

Libreria de Particulas para Terraria

*Implementacion desde cero, independiente de
cualquier libreria existente (clean room design)*

- ARQUITECTURA DATA-ORIENTED
- SHADERS HLSL DESDE CERO
- ASSETS COMPLETOS (TEXTURAS + SHADERS + AUDIO)
- SISTEMA DE LAYERS Y CULLING
- MULTIPLAYER Y SINCRONIZACION
- PERFORMANCE Y PROFILING
- CROSS-MOD COMPATIBILITY
- PUBLICACION EN STEAM WORKSHOP

PROPOSITO

Documento de contexto para alimentar a un modelo de IA capaz de generar
codigo funcional para una libreria de particulas de Terraria.

Audiencia: IA · Idioma: Espanol · Profundidad: Enciclopedia

Cobertura: tModLoader 1.4.4 · MonoGame · XNA · HLSL · C#

Indice

1. Introduccion y Contexto del Documento	9
1.1 Proposito del documento	9
1.2 Alcance y cobertura	9
1.3 Filosofia de implementacion: clean room design	10
1.4 Estructura del documento	10
2. Analisis Comparativo de Librerias Existentes	11
2.1 Por que estudiar lo existente antes de reescribir	11
2.2 ParticleLibrary (SnowyStarfall)	11
2.3 ParticleAnimationLib (Emafire003)	12
2.4 AnimLib (Ilemni)	12
2.5 Luminance (LucilleKarma)	12
2.6 TerrariaOverhaul (Mirsario)	12
2.7 CalamityModPublic (CalamityTeam)	13
2.8 Tabla comparativa	13
2.9 Conclusiones del analisis	13
3. Decisiones de Arquitectura para la Libreria Nueva	14
3.1 Principios rectores	14
3.2 Component-based vs Behavior-based	14
3.3 Structs vs clases	14
3.4 Arrays planos vs genericos	15
3.5 Layers como prioridades numericas	15
3.6 Spawn API generica	15
3.7 Sistema de configuracion	16
3.8 Resumen de decisiones	16
4. Estructura del Proyecto	17
4.1 Diseno de carpetas	17
4.2 build.txt	18
4.3 description.txt	18
4.4 MiParticleLib.cs - Clase Mod principal	19
5. Matematicas Subyacentes de las Particulas	19

5.1 Algebra vectorial 2D	19
5.2 Interpolacion lineal (lerp)	20
5.3 Curvas de interpolacion (easing)	20
5.4 Generacion de posiciones aleatorias	21
5.5 Sistemas de coordenadas: world vs screen	22
5.6 Time y framerate independence	22
6. MonoGame y XNA Graphics Pipeline	22
6.1 Que es MonoGame	22
6.2 SpriteBatch: el caballo de batalla	22
6.3 Begin y End: blend modes y save state	23
6.4 El pipeline de rendering de Terraria	23
6.5 Shaders en MonoGame	23
7. El Sistema Dust Vanilla de Terraria	24
7.1 Que es Dust y por que existe	24
7.2 API de Dust	25
7.3 Limitaciones criticas de Dust	26
7.4 Por que no usar Dust directamente	26
8. Por que Dust no es Suficiente: Casos de Uso Avanzados	26
8.1 Slash effects estilo Calamity	26
8.2 Auras orbitales	27
8.3 Trails continuos	27
8.4 Particulas con homing	27
8.5 Screenshake coordinado	27
9. Diseno Data-Oriented vs Object-Oriented	27
9.1 Que es data-oriented design (DOD)	27
9.2 SoA vs AoS	28
9.3 Compromiso: struct con fields relevantes	28
10. Structs vs Clases para Particulas	28
10.1 Diferencias tecnicas en C#	28
10.2 Diseno del struct ParticleData	30
10.3 Component flags	31
11. Buffers Pre-asignados y Gestion de Memoria	32

11.1 Por que pre-asignar	32
11.2 Implementacion del buffer	32
11.3 Estrategia round-robin	34
11.4 Capacidad configurable	34
12. Sistema de Components	34
12.1 Diseno de IComponent	34
12.2 Componentes implementados	35
13. Sistema de Emitters	36
13.1 Que es un emitter	36
13.2 Clase base Emitter	37
13.3 Implementacion de ShapeDescriptor	38
14. ParticleManager: Orquestador Central	39
14.1 Responsabilidades del manager	39
15. Render Pipeline y Sistema de Layers	42
15.1 Layers como prioridades numericas	42
15.2 Hook en el pipeline de tModLoader	43
16. Culling y Optimizaciones de Performance	44
16.1 Frustum culling	44
16.2 Batching por blend mode	44
16.3 SpriteSortMode.Deferred vs Immediate	45
16.4 Texture atlas consideration	45
17. Multiplayer y Sincronizacion	45
17.1 El problema de la sincronizacion	45
17.2 Enfoque adoptado: client-side only	45
17.3 Hooks multiplayer-safe	45
17.4 Sincronizacion opcional para eventos criticos	46
18. Hooks de tModLoader para VFX	46
18.1 ModSystem hooks	46
18.2 GlobalItem hooks para VFX de items	46
18.3 GlobalProjectile hooks	47
18.4 ModPlayer hooks	47

19. Implementacion de ModSystem, GlobalItem, GlobalProjectile	48
19.1 ParticleSystem (ModSystem principal)	48
19.2 GlobalProjectile con trail automatico	49
20. PlayerLayer y Custom Drawing	50
20.1 Que es PlayerLayer	50
20.2 PlayerDrawLayer para auras	50
20.3 ModPlayer con estado de aura	51
21. Asset Pipeline: Carga de Texturas	51
21.1 Como tModLoader carga texturas	51
21.2 Registro centralizado de texturas	52
22. Asset Pipeline: Carga de Shaders	52
22.1 Compilacion de shaders .fx a .xnb	52
22.2 Carga de shaders en tModLoader	53
22.3 Aplicar un shader en SpriteBatch	53
23. Shaders HLSL: Introduccion	54
23.1 Que es HLSL	54
23.2 Estructura de un shader .fx	54
23.3 Compilacion a .xnb	55
24. Shader de Quad Particle	55
24.1 Vertex format custom	55
24.2 Codigo HLSL completo	56
25. Shader de Instanced Particles (GPU)	58
25.1 Que es instancing	58
25.2 Codigo HLSL de instanced shader	59
26. Shader de Formas Procedurales (SDF)	60
26.1 Que es SDF	60
26.2 Shader Shape.fx	61
27. Shader de Bloom	62
27.1 Que es bloom	62
27.2 Bloom shader HLSL	63
28. Shader de Chromatic Aberration	64

28.1 Que es chromatic aberration	64
28.2 Shader HLSL	64
29. Shader de Vignette	64
29.1 Vignette effect	64
30. Shader de Shockwave	65
30.1 Shockwave effect	65
31. Shader de Distortion / Heat Haze	66
31.1 Distorsion por calor	66
32. Shader de Color Grading	67
32.1 Color grading con LUT	67
33. Shader de Radial Blur	67
33.1 Radial blur effect	67
34. Compilacion de Shaders: EasyXnb y MGCB	68
34.1 EasyXnb.exe	68
34.2 MGCB (MonoGame Content Builder)	68
34.3 Script de build automatizado	69
35. Lista Completa de Texturas Necesarias	69
35.1 Texturas built-in de la libreria	69
35.2 Texturas para shaders	70
35.3 Texturas para UI interna	70
35.4 Especificaciones tecnicas de cada textura	70
36. Lista Completa de Shaders Necesarios	71
36.1 Shaders core	71
36.2 Shaders de screen effect	71
37. Lista Completa de Sonidos Necesarios	72
37.1 Sonidos built-in	72
37.2 Audio para efectos pre-construidos	72
38. Generacion Procedural de Texturas con Python	72
38.1 Script para generar todas las texturas	72
38.2 Ejemplo: generar SoftGlow.png	73
38.3 Ejemplo: generar Trail.png	73

39. Diseno de Texturas por Tipo	73
39.1 Texturas radiales (SoftGlow, Aura, Ring)	73
39.2 Texturas con shape definida (Star, Spark, Slash)	73
39.3 Texturas con animacion (spritesheets)	74
40. Sistema de Configuracion del Mod	74
40.1 ModConfig de tModLoader	74
41. Sistema de Logging	75
41.1 Por que logging estructurado	75
42. Debugging y Profiling	76
42.1 Debug overlay	76
42.2 Profiling con Stopwatch	76
43. Testing en Multiplayer	77
43.1 Setup de test multiplayer	77
43.2 Checklist de multiplayer-safe	77
44. Performance Benchmarks	78
44.1 Metricas objetivo	78
44.2 Tabla de benchmarks esperados	78
45. Cross-Mod Compatibility	78
45.1 Mod Calls API	78
45.2 WeakReferences	79
46. Documentacion de la API Publica	79
46.1 XML documentation comments	79
46.2 API publica principal	80
47. Publicacion en Steam Workshop	81
47.1 Preparacion para publicar	81
47.2 Proceso de publicacion	81
47.3 Update workflow	81
48. Mantenimiento y Versionado	81
48.1 Semantic versioning	81
48.2 Backwards compatibility	81
48.3 Branching strategy	81

49. Roadmap y Features Futuras	82
49.1 Features planeadas para v0.2	82
49.2 Features planeadas para v0.3	82
49.3 Features planeadas para v1.0	82
50. Apendice A: Snippets Completos deCodigo	83
50.1 Spawn de explosion basica	83
50.2 Aura orbital persistente	84
51. Apendice B: Lista de IDs de Terraria Utiles	85
51.1 ItemUseStyleID	85
51.2 SoundID utiles	85
52. Apendice C: Glosario	86
53. Apendice D: Recursos y Referencias	87
53.1 Documentacion oficial	87
53.2 Librerias de referencia	87
53.3 Comunidad	88
53.4 Herramientas	88
53.5 Conclusion final	88

1. Introduccion y Contexto del Documento

1.1 Proposito del documento

Este documento ha sido elaborado como referencia tecnica enciclopedica para alimentar el contexto de un modelo de inteligencia artificial. Su mision es proporcionar todo el conocimiento necesario para que una IA pueda generar, sin asistencia humana adicional, el codigo fuente completo de una libreria de particulas para el videojuego Terraria, ejecutandose sobre tModLoader 1.4.4. La libreria objetivo debe ser funcionalmente equivalente a las soluciones existentes (como ParticleLibrary de SnowyStarfall o sistemas equivalentes en mods grandes como Calamity y TerrariaOverhaul), pero escrita desde cero con una arquitectura propia y licenciamiento libre (MIT o Apache 2.0).

El enfoque es deliberadamente enciclopedico: en lugar de resumir conceptos, se exponen con detalle extenso. Cada decision de diseno se justifica, cada fragmento de codigo se explica linea por linea, y cada asset necesario (texturas, shaders, sonidos) se documenta con sus dimensiones, formato, ruta y proposito. La idea es que un desarrollador (humano o IA) que lea este documento de principio a fin termine con la capacidad de implementar la libreria completa sin tener que buscar informacion adicional en otras fuentes.

El destinatario primario es un modelo de IA. Esto implica que el documento prioriza la precision tecnica sobre la elegancia literaria, usa terminologia exacta (nombres de clases, metodos, hooks, namespaces) sin rodeos, y estructura la informacion de forma que pueda ser referenciada por seccion cuando se necesite un detalle concreto. Sin embargo, el tono sigue siendo pedagogico: se introducen los conceptos gradualmente, se justifica por que existen las cosas antes de explicar como funcionan, y se incluyen comparaciones con enfoques alternativos para que la IA pueda elegir el mas apropiado segun el contexto de uso.

1.2 Alcance y cobertura

El documento cubre todo el ciclo de vida del desarrollo de una libreria de particulas, desde la investigacion previa de soluciones existentes hasta la publicacion en Steam Workshop y el mantenimiento a largo plazo. Esto incluye: analisis comparativo de librerias de referencia, decisiones de arquitectura, matematicas subyacentes (algebra vectorial, interpolacion, blending modes), diseno de la API publica, implementacion del rendering pipeline con MonoGame/XNA, escritura de shaders HLSL para GPU, generacion procedural de assets, sistema de layers y culling, sincronizacion multiplayer, optimizacion de performance, debugging y profiling, documentacion de la API, cross-mod compatibility, y publicacion.

No se asume conocimiento previo del lector sobre librerias de particulas especificas de Terraria. Si se asume familiaridad con programacion en C#, conceptos basicos de graficos por computadora (vertex shaders, pixel shaders, texturas, blending), y la API basica de tModLoader (ModItem, ModProjectile, ModSystem, GlobalItem, ModPlayer). Cuando se introduce un concepto avanzado, se define brevemente antes de usarlo.

1.3 Filosofia de implementacion: clean room design

Este documento promueve un enfoque de clean room design (diseno en cuarto limpio). Esto significa que, aunque se estudian las soluciones existentes para entender sus patrones y limitaciones, toda la implementacion se escribe desde cero sin copiar codigo fuente ni assets de terceros. La razon es doble: por un lado, evita problemas de licenciamiento (la libreria ParticleLibrary usa GPL v3, que es copyleft fuerte; TerrariaOverhaul tambien es GPL v3); por otro, fuerza a entender profundamente cada componente en lugar de copiar por imitacion.

Es importante reconocer que en el dominio de las librerias de particulas para Terraria, gran parte del codigo se parecera naturalmente entre implementaciones. Esto se debe a que la API de tModLoader/MonoGame fuerza ciertas firmas, las matematicas de particulas son universales (vectores, interpolacion, blending), los formatos de vertices para shaders estan dictados por lo que el GPU soporta, y los patrones de rendering (instancing, batching) son estandar de la industria. Por tanto, cualquier implementacion de una libreria de particulas para Terraria va a parecerse un 70-80% a cualquier otra, sin importar quien la escriba. Esto no constituye derivacion legal siempre y cuando el codigo sea claramente original en estilo, nombres, organizacion y diseno detallado.

Para asegurar originalidad, este documento propone decisiones de diseno concretas que difieren de las soluciones existentes: usar un sistema component-based inspirado en ECS en lugar del patron Behavior<T> de ParticleLibrary; usar arrays planos con discriminadores de tipo en lugar de buffers genericos; usar strings o prioridades numericas en lugar de un enum Layer hardcoded; ofrecer una API de spawn generica basada en ShapeDescriptor en lugar de metodos especificos (SpawnBox, SpawnCircle, etc.). Estas diferencias estructurales hacen que el codigo sea claramente independiente, no derivado.

1.4 Estructura del documento

El documento se organiza en 53 capitulos agrupados en 9 partes logicas. La parte I (capitulos 1-4) cubre contexto, investigacion de soluciones existentes y decisiones de arquitectura. La parte II (capitulos 5-9) cubre los fundamentos matematicos y de rendering sobre los que se construye cualquier sistema de particulas. La parte III (capitulos 10-16) detalla el diseno e implementacion del nucleo de la libreria: structs de datos, buffers, behaviors, emitters, render pipeline y optimizaciones. La parte IV (capitulos 17-20) cubre integracion con tModLoader via hooks y systems. La parte V (capitulos 21-22) cubre el asset pipeline. La parte VI (capitulos 23-34) es una guia extensa de shaders HLSL con codigo completo. La parte VII (capitulos 35-39) cubre la lista exhaustiva de assets necesarios y su generacion procedural. La parte VIII (capitulos 40-44) cubre configuracion, debugging, profiling y testing. La parte IX (capitulos 45-49) cubre cross-mod compatibility, documentacion y publicacion. Finalmente, los apendices (capitulos 50-53) recopilan snippets completos de codigo, listas de IDs, glosario y referencias.

Cada capitulo sigue una estructura consistente: introduccion conceptual, justificacion de decisiones, codigo o especificacion tecnica, y conclusion o transicion al siguiente capitulo. Los bloques de codigo estan formateados con fuente monoespaciada y fondo claro para distinguirlos del texto prosa. Las tablas se usan para comparar opciones o listar elementos categorizados. Las notas (en cursiva y con una flecha) destacan advertencias o detalles importantes que no deben pasarse por alto.

2. Analisis Comparativo de Librerias Existentes

2.1 Por que estudiar lo existente antes de reescribir

Antes de disenar una libreria nueva conviene entender que soluciones existen, que problemas resuelven, cuales son sus limitaciones, y por que la comunidad las usa o no las usa. Esto cumple tres funciones: primero, evita repetir errores ya cometidos por otros; segundo, identifica features que los usuarios esperan encontrar en una libreria de este tipo; tercero, ayuda a definir un nicho diferenciador para la nueva libreria (no tiene sentido publicar otra libreria identica a las existentes).

En el ecosistema de modding de Terraria hay cuatro librerias/mods que merecen estudio detallado: ParticleLibrary de SnowyStarfall, ParticleAnimationLib de Emafire003 (originalmente para Minecraft Fabric pero con patrones transferibles), AnimLib de Ilemni, y Luminance de LucilleKarma. Adicionalmente, mods grandes como TerrariaOverhaul de Mirsario y CalamityModPublic de CalamityTeam implementan sistemas de particulas internos que no son librerias publicas pero que sirven como referencia de patrones avanzados. Este capitulo analiza cada uno en detalle.

2.2 ParticleLibrary (SnowyStarfall)

ParticleLibrary es la libreria de particulas mas popular y madura del ecosistema tModLoader. Su repositorio GitHub esta en <https://github.com/SnowyStarfall/ParticleLibrary>, su Steam Workshop tiene ID 2760520795, y su licencia es GNU GPL v3 (copyleft fuerte). La version actual a la fecha de esta investigacion es 3.2.1.0, y mods como Mod of Redemption, Shadows of Abaddon y Lunar Veil la usan como dependencia.

La libreria ofrece tres versiones paralelas del API que coexisten en el repositorio: V1 (obsoleta, sera eliminada), V2 (orientada a objetos, con clase Particle base y metodos Spawn/Update/Draw/Death), y V3 (data-oriented, con structs ParticleInfo y clases Behavior<T> genericas). Esta coexistencia refleja la evolucion del diseno: V1 era simple pero ineficiente, V2 introdujo orientacion a objetos con lifetime methods, V3 migro a data-oriented design con structs y buffers pre-asignados para mejorar cache locality y reducir allocations.

Las features clave de ParticleLibrary incluyen: buffers pre-asignados con tamaño configurable (evita GC pressure), sistema de Layers con mas de 20 capas de render distintas, emitters con cuatro formas geometricas (Box, Circle, HollowBox, HollowCircle), soporte para GPU particles via instancing, sistema UI interno con 17 texturas, helpers de conversion (LibUtilities), y una herramienta EasyXnb.exe para compilar shaders sin instalar MonoGame Content Pipeline completo.

Las limitaciones que presenta son: la licencia GPL v3 "contamina" cualquier mod que la use como dependencia (tecnicamente, no en la practica); el API V3 requiere pensar en data-oriented design lo cual tiene curva de aprendizaje; el sistema de Layers con enum hardcoded es inflexible; y la documentacion, aunque existen ejemplos, no es exhaustiva. Estos puntos definen oportunidades de mejora para una libreria nueva.

2.3 ParticleAnimationLib (Emafire003)

ParticleAnimationLib es una libreria originalmente disenada para Minecraft Fabric (no Terraria), pero su analisis es valioso porque ilustra un enfoque de API distinto. Su repositorio esta en <https://github.com/Emafire003/ParticleAnimationLib> y su licencia es MIT (permisiva). Esta inspirada en el plugin EffectLib de Bukkit.

La libreria ofrece efectos pre-construidos como objetos: AnimatedSphere, Sphere, Cuboid, Vortex, Cone, Donut (3D), y Arc, Line, AnimatedCircle, ColoredImage, Text (2D). Cada efecto se instancia como un objeto y se ejecuta con `run()` o `runFor(ticks)`. Tambien ofrece un patron Builder para configurar parametros. Esto contrasta con el enfoque de ParticleLibrary donde el usuario debe componer behaviours atomicos.

La leccion aprendida de ParticleAnimationLib es que existe demanda para una API de alto nivel que ofrezca efectos pre-empaquetados (esferas, conos, vortex) ademas de la API de bajo nivel para partículas individuales. Una libreria nueva idealmente ofreceria ambos niveles: la API atomica para usuarios avanzados y una galeria de efectos pre-construidos para usuarios que quieren resultados rapidos.

2.4 AnimLib (Ilemni)

AnimLib es una libreria de animaciones (no de particulas como tales, pero relacionada) para tModLoader. Su repositorio esta en <https://github.com/Ilemni/AnimLib>. Su proposito es proveer un framework para crear animaciones de spritesheet complejas en NPCs, projectiles y player layers.

Aunque no es competidor directo de una libreria de particulas, AnimLib es relevante porque comparte problemas: como gestionar el ciclo de vida de frames, como sincronizar animaciones en multiplayer, como integrar con el rendering pipeline de tModLoader. Estudiar AnimLib ayuda a entender patrones de gestion de estado animado que aplican tambien a particulas con ciclo de vida complejo (por ejemplo, particulas que cambian de textura al morir).

2.5 Luminance (LucilleKarma)

Luminance es una libreria general-purpose para mods de Terraria. Su repositorio esta en <https://github.com/LucilleKarma/Luminance>. Es relevante porque Calamity (el mod mas grande) usa partes de ella. Ofrece utilidades para VFX, gestion de assets, y serializacion.

La leccion de Luminance es que una libreria de utilidades transversales puede complementar a una libreria de particulas. Por ejemplo, Luminance podria manejar la serializacion de configuracion de particulas, mientras la libreria de particulas maneja el rendering. Separar responsabilidades sigue el principio Unix: haz una cosa y hazla bien.

2.6 TerrariaOverhaul (Mirsario)

TerrariaOverhaul es un mod masivo que mejora el "feel" del juego: animaciones de combate, screen shake, particles mejoradas, camera roll, blood physics, weapon sway. Su repositorio esta en <https://github.com/Mirsario/TerrariaOverhaul> y su licencia es GPL v3 (con excepciones para algunos archivos MIT como FastNoiseLite.cs).

Aunque no es una libreria de particulas como tal, TerrariaOverhaul contiene un sistema interno de particulas muy avanzado que sirve como referencia de patrones profesionales. En particular, muestra como integrar particulas con sistemas de camara (screen shake, zoom dinamico), como usar PlayerLayer para dibujar efectos sobre el jugador, y como estructurar namespaces en Core/Common/Content/Utilities siguiendo la heuristica "usaria X sin Y?".

2.7 CalamityModPublic (CalamityTeam)

Calamity es el mod mas grande de Terraria. Su mirror publico esta en <https://github.com/CalamityTeam/CalamityModPublic>. No es una libreria de particulas, pero contiene implementaciones internas que sirven de referencia para VFX avanzados: boss intros dramaticas, screen shaders por boss, slash effects en armas melee, stealth meter visual para la clase Rogue, y Luminance mode (difficulty) que anade chromatic aberration y bloom amplificado.

Estudiar Calamity es util para entender el extremo superior del espectro de complejidad: como manejar miles de particulas simultaneas en un boss fight sin perder performance, como coordinar VFX entre cliente y servidor, y como estructurar carpetas por clase de dano (Items/Melee/, Items/Ranged/, Items/Magic/, Items/Summon/, Items/Rogue/).

2.8 Tabla comparativa

Libreria/Mod	Licencia	Lenguaje	Estado	Caso de uso
ParticleLibrary	GPL v3	C#	Activo v3.2	Libreria de particulas principal
ParticleAnimationLib	MIT	Java	Beta	Efectos pre-empaquetados (Minecraft)
AnimLib	Por verificar	C#	Activo	Framework de animaciones
Luminance	Por verificar	C#	Activo	Utilidades generales
TerrariaOverhaul	GPL v3	C#	v5 ~80%	Mod de mejora de feel
CalamityModPublic	Mixto	C#	Activo	Mod masivo de contenido

2.9 Conclusiones del analisis

Del analisis comparativo se extraen las siguientes conclusiones que guian el diseno de una libreria nueva: (1) existe demanda real de una libreria de particulas con licencia permisiva (MIT o Apache 2.0), dado que las dos opciones principales para Terraria (ParticleLibrary y TerrariaOverhaul) son GPL v3; (2) el enfoque data-oriented de ParticleLibrary V3 es el camino correcto en terminos de performance, pero su API puede simplificarse; (3) ofrecer una galeria de efectos pre-empaquetados (como hace ParticleAnimationLib) agrega valor significativo para usuarios que quieren resultados rapidos; (4) el sistema de Layers hardcoded es inflexible y conviene reemplazarlo por prioridades numericas; (5) la herramienta EasyXnb para compilar shaders es valiosa y conviene incluir una equivalente; (6) la documentacion debe ser exhaustiva, con ejemplos por cada feature.

3. Decisiones de Arquitectura para la Libreria Nueva

3.1 Principios rectores

Antes de detallar la arquitectura, conviene establecer los principios que guian cada decision. Estos principios sirven como criterio para resolver disputas de diseno: cuando dos enfoques parecen equivalentes, gana el que mejor satisface estos principios. Los siete principios rectores son: simplicidad de API, performance por defecto, originalidad de implementacion, flexibilidad de extension, compatibilidad con multiplayer, documentacion exhaustiva, y licenciamiento libre.

Simplicidad de API significa que el usuario final (un modder escribiendo codigo que usa la libreria) deberia poder crear una partícula básica con menos de 10 líneas de código. Performance por defecto significa que la configuración inicial de la libreria debe ser la más eficiente posible, sin requerir tuning manual. Originalidad de implementacion significa que cada decision de diseno debe diferir estructuralmente de las librerias existentes para evitar problemas de licenciamiento. Flexibilidad de extension significa que un usuario avanzado puede anadir comportamientos nuevos sin tocar el código core. Compatibilidad con multiplayer significa que todas las APIs deben ser seguras para uso en servidor dedicado. Documentacion exhaustiva significa que cada clase y metodo publico tiene documentacion XML. Licenciamiento libre significa MIT o Apache 2.0.

3.2 Component-based vs Behavior-based

ParticleLibrary V3 usa un patron Behavior<T> donde cada tipo de partícula tiene una clase Behavior asociada que define su logica. Esto es elegante pero acopla la definicion de la partícula con su comportamiento. La alternativa, que esta libreria adopta, es un diseno component-based inspirado en Entity-Component-System (ECS): cada partícula es un struct ParticleData que contiene components flags indicando que behaviors tiene activos, y un ParticleProcessor itera sobre todos los componentes aplicando logica.

La ventaja del enfoque component-based es que los behaviors son reutilizables: una partícula puede tener los componentes Gravity, FadeOut y HomingTarget sin necesidad de definir una clase nueva. La desventaja es que la API es ligeramente mas verbosa para casos simples. Para casos complejos (partículas con 5+ behaviors), el enfoque component-based escala mucho mejor.

3.3 Structs vs clases

Las partículas se representan como structs (value types) en lugar de clases (reference types). Esto tiene implicaciones de performance criticas: structs alojados en arrays contiguos tienen mejor cache locality que clases (que se asignan en el heap y se accede via referencias). Para sistemas que iteran sobre miles de partículas por frame, esto puede traducirse en 2-5x mejor rendimiento. La desventaja es que structs no soportan herencia, pero el diseno component-based elimina la necesidad de herencia.

El struct ParticleData debe mantenerse pequeno (idealmente menos de 64 bytes) para que quepa en una linea de cache tipica. Los campos a incluir son: Position (Vector2, 8 bytes), Velocity (Vector2, 8 bytes),

Rotation (float, 4 bytes), Scale (Vector2, 8 bytes), Color (Color, 4 bytes o 16 bytes segun implementacion), TimeLeft (int, 4 bytes), Duration (int, 4 bytes), TextureId (ushort, 2 bytes), ComponentFlags (ulong, 8 bytes), UserData (float[4], 16 bytes para datos custom). Total aproximado: 66-82 bytes, dependiendo de implementacion de Color y campos adicionales.

3.4 Arrays planos vs genericos

ParticleLibrary V3 usa ParticleBuffer<T> genericos donde T es el tipo de Behavior. Esto implica que cada tipo de partícula tiene su propio buffer, lo que facilita el dispatch pero fragmenta la memoria. La alternativa, adoptada por esta libreria, es un unico array plano de ParticleData donde todas las partículas conviven, con un campo TextureId (ushort) que identifica que textura usar. Para aplicar logica especifica de tipo, se usa el campo ComponentFlags como bitmask.

La ventaja del array plano es que iterar sobre todas las partículas es un solo loop, no N loops. Esto mejora cache locality y reduce overhead de dispatch. La desventaja es que aplicar logica especifica requiere chequear ComponentFlags en cada iteracion. Para partículas con muchos behaviors, esto puede ser menos eficiente que dispatch virtual directo. Sin embargo, en la practica, la mayoría de partículas tienen 1-3 behaviors activos, y el chequeo de bitmask es extremadamente rapido (una operacion AND a nivel de bits).

3.5 Layers como prioridades numericas

ParticleLibrary usa un enum Layer con mas de 20 valores hardcoded (BeforeWalls, BeforeTiles, BeforeNPCs, etc.). Esto es inflexible: si un usuario quiere una capa entre dos existentes, no puede. La alternativa es usar prioridades numericas (float o int) donde valores mas bajos significan render antes. El usuario puede especificar cualquier valor, y la libreria ordena las partículas por prioridad antes de renderizar.

Para mantener compatibilidad mental con el modelo de tModLoader, se definen constantes pre-establecidas: LayerPriority.BeforeWalls = 100, LayerPriority.BeforeTiles = 200, LayerPriority.BeforeNPCs = 300, etc. El usuario puede usar las constantes o especificar un valor custom como 250 para renderizar entre BeforeWalls y BeforeTiles.

El ordenamiento por prioridad puede hacerse una vez por frame (no por partícula), ya que la mayoría de partículas mantienen su layer durante toda su vida. Si una partícula cambia de layer, se marca como dirty y se reordena solo ese subset. Esto mantiene el overhead minimo.

3.6 Spawn API generica

En lugar de ofrecer metodos especificos SpawnBox, SpawnCircle, SpawnHollowBox, SpawnHollowCircle (como hace ParticleLibrary), esta libreria ofrece una API generica basada en un struct ShapeDescriptor. Un ShapeDescriptor define una forma geometrica y sus parametros, y el metodo Spawn(shape, count, particleData) se encarga de posicionar las partículas segun la forma.

```
public readonly struct ShapeDescriptor
{
    public ShapeType Type { get; }
    public Vector2 Size { get; }      // width/height para box, radius para circle
}
```



```

public bool Hollow { get; }
public float Rotation { get; }    // para formas orientadas

// Constructores estaticos para formas comunes
public static ShapeDescriptor Box(float w, float h) =>
    new(ShapeType.Box, new Vector2(w, h), false, 0f);
public static ShapeDescriptor Circle(float r) =>
    new(ShapeType.Circle, new Vector2(r, r), false, 0f);
public static ShapeDescriptor HollowBox(float w, float h) =>
    new(ShapeType.Box, new Vector2(w, h), true, 0f);
public static ShapeDescriptor HollowCircle(float r) =>
    new(ShapeType.Circle, new Vector2(r, r), true, 0f);
public static ShapeDescriptor Cone(float length, float angle) =>
    new(ShapeType.Cone, new Vector2(length, angle), false, 0f);
public static ShapeDescriptor Sphere(float r) =>
    new(ShapeType.Sphere, new Vector2(r, r), false, 0f); // 3D proyectada a
2D
    public static ShapeDescriptor Vortex(float r, float turns) =>
        new(ShapeType.Vortex, new Vector2(r, turns), false, 0f);
}

```

Esta API es mas flexible: anadir una forma nueva (ej: espiral, estrella, custom spline) solo requiere anadir un caso al switch en el metodo Spawn, sin tocar la API publica. Ademas, el usuario puede pasar ShapeDescriptor custom para formas arbitrarias.

3.7 Sistema de configuracion

La libreria expone una configuracion al usuario via ModConfig de tModLoader. Esto permite al jugador final ajustar parametros como: maximo de particulas activas (para hardware modesto), calidad de blending (alpha vs additive), activar/desactivar layers especificas, y logging verbose para debugging. La configuracion se carga en OnModLoad y se aplica en caliente cuando el usuario cambia un valor.

3.8 Resumen de decisiones

Aspecto	ParticleLibrary	Esta libreria	Razon
Paradigma	Behavior<T>	Component-based	Reusabilidad de behaviors
Tipo de dato	Struct ParticleInfo	Struct ParticleData	Cache locality
Buffer	Generic ParticleBuffer<T>	Array plano unico	Menos fragmentacion
Layers	Enum 20+ valores	Prioridad numerica	Flexibilidad
Spawn	Metodos especificos	ShapeDescriptor generico	Extensibilidad
Config	Hardcoded	ModConfig	Customizacion usuario
Licencia	GPL v3	MIT	Permisiva

4. Estructura del Proyecto

4.1 Diseno de carpetas

La estructura de carpetas del proyecto refleja las decisiones de arquitectura del capítulo anterior y sigue la heurística de TerrariaOverhaul: Core para funcionalidad engine-like, Common para logica general, Content para contenido de juego (en este caso ejemplos y presets), y Utilities para helpers mod-agnostic. La organizacion no se basa en que tipos contiene cada carpeta, sino en que area de diseno de juego representa.

```
MiParticleLib/
├─ build.txt
├─ description.txt
├─ icon.png                # 256x256 para Steam Workshop
├─ MiParticleLib.csproj
├─ MiParticleLib.cs        # Clase Mod principal
├─ MiParticleLibConfig.cs  # ModConfig
├─ LICENSE                 # MIT
├─ README.md
├─ CHANGELOG.md
├─ Core/                   # Engine-like
│   ├── ParticleData.cs    # Struct principal
│   ├── ParticleManager.cs # Singleton manager
│   ├── ParticleBuffer.cs  # Array plano + gestion
│   ├── ParticleProcessor.cs # Itera y aplica behaviors
│   ├── ParticleRenderer.cs # Render via SpriteBatch
│   └─ Components/
│       ├── IComponent.cs  # Interface base
│       ├── GravityComponent.cs
│       ├── FadeOutComponent.cs
│       ├── HomingComponent.cs
│       ├── ScaleAnimationComponent.cs
│       ├── RotationComponent.cs
│       ├── ColorShiftComponent.cs
│       └─ TrailComponent.cs
│   └─ Emitters/
│       ├── Emitter.cs      # Clase base
│       ├── ShapeDescriptor.cs
│       └─ ShapeType.cs
│   └─ Layers/
│       ├── LayerPriority.cs # Constantes numericas
│       └─ LayerRegistry.cs
│   └─ Shaders/
│       ├── ShaderLoader.cs
│       └─ ShaderIds.cs
│   └─ Culling/
│       ├── Culler.cs
│       └─ CameraBounds.cs
├─ Common/
│   ├── ModSystem/
│   │   ├── ParticleSystem.cs # ModSystem principal
│   │   └─ DebugSystem.cs
│   ├── GlobalItems/
│   │   └─ GlobalParticleItem.cs # Hooks para items
│   └─ GlobalProjectiles/
│       └─ GlobalParticleProjectile.cs
└─ Content/
    ├── Textures/
    │   ├── Default.png      # 32x32 estrella
    │   └─ WhitePixel.png    # 1x1
```

```

├── SoftGlow.png          # 64x64 radial
├── Spark.png             # 16x16
├── Smoke.png             # 32x32
├── Trail.png             # 64x8 degradado
├── Effects/              # Shaders .xnb
│   ├── ParticleShader.xnb
│   ├── InstancedShader.xnb
│   └── ShapeShader.xnb
├── Sounds/
│   ├── Pop.wav
│   └── Whoosh.wav
├── Presets/              # Efectos pre-construidos
│   ├── ExplosionPreset.cs
│   ├── AuraPreset.cs
│   ├── VortexPreset.cs
│   └── TrailPreset.cs
├── Utilities/
│   ├── ColorUtils.cs
│   ├── VectorUtils.cs
│   ├── FastList.cs
│   └── SpriteBatchExtensions.cs
└── Documentation/
    └── API.md             # Referencia de API publica

```

4.2 build.txt

El archivo build.txt define los metadatos de compilación del mod. Para una librería, es crucial configurar correctamente modReferences (ninguna, idealmente), side (Both para que funcione en cliente y servidor), y buildIgnore para excluir archivos de desarrollo del build final.

```

author = Tu Nombre
version = 0.1.0
displayName = Mi Particle Lib
homepage = https://github.com/tu-usuario/mi-particle-lib
modReferences =
dllReferences =
side = Both
buildIgnore = *.csproj.user, *.sln, obj/*, bin/*, .vs/*, .vscode/*,
               *.md, Documentation/*, *.gitignore, *.gitattributes,
               *.tt, *.json, Properties/*
includeSource = true          # Librería: incluir source para debug
includePDB = true            # Incluir PDB para stack traces
hideCode = false             # Source visible (librería)
hideResources = false        # Resources accessible

```

4.3 description.txt

La descripción es lo que ve el usuario en el menú de mods in-game. Para una librería, debe comunicar claramente que no añade gameplay por sí misma, sino que es un API para otros mods. También debe creditar inspiraciones.

Mi Particle Lib

Una librería de partículas performante y customizable para tModLoader 1.4.4. Esta librería NO añade contenido por sí misma. Es un API para que otros mods la usen.

Features:

- Sistema component-based para behaviors reutilizables

- Buffers pre-asignados para performance
- 8 formas geometricas pre-construidas (Box, Circle, Hollow, Cone, Sphere, Vortex, etc.)
- Shaders HLSL para GPU instancing
- ModConfig para ajustes del usuario
- Multiplayer-safe por diseno

Licencia: MIT. Inspirada en el trabajo de SnowyStarfall (ParticleLibrary) y Mirsario (TerrariaOverhaul), pero escrita desde cero con arquitectura propia.

Source: <https://github.com/tu-usuario/mi-particle-lib>

4.4 MiParticleLib.cs - Clase Mod principal

```
using Terraria.ModLoader;

namespace MiParticleLib
{
    public class MiParticleLib : Mod
    {
        public static MiParticleLib Instance { get; private set; }
        public static ParticleConfig Config { get; private set; }

        public override void Load()
        {
            Instance = this;
            // Inicializacion del sistema de particulas
            ParticleManager.Initialize();
            ShaderLoader.LoadAll(Mod);
        }

        public override void Unload()
        {
            ParticleManager.Shutdown();
            Instance = null;
        }

        public override void PostSetupContent()
        {
            // Despues de que todos los mods carguen - util para cross-mod
        }
    }
}
```

5. Matematicas Subyacentes de las Particulas

5.1 Algebra vectorial 2D

Toda partícula en Terraria opera en un espacio 2D. Su posición, velocidad, aceleración y dirección se representan con Vector2 (Microsoft.Xna.Framework.Vector2 o System.Numerics.Vector2). Es esencial dominar las operaciones básicas: suma, resta, multiplicación escalar, normalización, longitud, producto punto, producto cruz (en 2D devuelve un escalar), rotación, e interpolación lineal (lerp).

Una operacion frecuente es "rotar un vector por un angulo". En XNA vanilla Vector2 no tiene un metodo Rotate nativo, pero tModLoader anade un metodo de extension RotatedBy(angle) que es lo que se usa habitualmente. Alternativamente se puede rotar manualmente con coseno/seno. Para partículas que orbitan un punto, se calcula el vector del centro a la partícula, se rota por un delta angular por frame, y se vuelve a sumar el centro. Para partículas con homing, se calcula el vector al target, se normaliza, y se interpola la velocidad actual hacia esa direccion.

5.2 Interpolacion lineal (lerp)

La interpolacion lineal, o lerp, es fundamental para animar cualquier propiedad de una partícula a lo largo de su vida. La formula es: $result = a + (b - a) * t$, donde t va de 0 a 1. Para colores, lerp interpola cada canal RGB por separado. Para vectores, interpola X e Y por separado. Para escalares, es directo. El parametro t se calcula tipicamente como TimeLeft / Duration, que va de 1 (recien nacida) a 0 (a punto de morir).

```
// Lerp escalar
public static float Lerp(float a, float b, float t) =>
    a + (b - a) * t;

// Lerp Vector2
public static Vector2 Lerp(Vector2 a, Vector2 b, float t) =>
    a + (b - a) * t;

// Lerp Color
public static Color LerpColor(Color a, Color b, float t) =>
    new Color(
        (byte)Lerp(a.R, b.R, t),
        (byte)Lerp(a.G, b.G, t),
        (byte)Lerp(a.B, b.B, t),
        (byte)Lerp(a.A, b.A, t)
    );

// Uso tipico: interpolar color durante vida de partícula
float t = particle.TimeLeft / (float)particle.Duration; // 1 -> 0
Color current = LerpColor(particle.StartColor, particle.EndColor, 1f - t);
```

5.3 Curvas de interpolacion (easing)

Lerp lineal es correcto pero visualmente "plano". Las curvas de easing anaden carcter: ease-in (lento al inicio, rapido al final), ease-out (rapido al inicio, lento al final), ease-in-out (lento en ambos extremos). Para partículas, las mas usadas son: ease-out para fade-out (la partícula se desvanece lentamente al final), ease-in para fade-in (aparece gradualmente), y ease-in-out para movimientos oscilantes.

Easing	Formula	Uso tipico
Linear	t	Movimiento constante
EaseInQuad	t*t	Aceleracion
EaseOutQuad	1-(1-t)*(1-t)	Desaceleracion, fade-out
EaseInOutQuad	t<0.5 ? 2*t*t : 1-(-2*t+2)*(-2*t+2)/2	Ambos
EaseInCubic	t*t*t	Aceleracion fuerte

Easing	Formula	Uso tipico
EaseOutCubic	$1 - \text{Math.Pow}(1-t, 3)$	Desaceleracion fuerte
EaseInExpo	$\text{Math.Pow}(2, 10*(t-1))$	Aparicion dramatica
EaseOutExpo	$1 - \text{Math.Pow}(2, -10*t)$	Desaparicion dramatica
EaseOutBounce	Compleja (ver codigo)	Efecto rebote

5.4 Generacion de posiciones aleatorias

Para spawnear partículas en una forma, se necesita generar posiciones aleatorias dentro de esa forma. Las formas mas comunes son: caja solida, circulo solido, caja hueca (solo bordes), circulo hueco (solo circunferencia). Cada una tiene su algoritmo optimo.

```
// Box solida: generar X e Y aleatorios dentro del rango
public static Vector2 RandomPointInBox(float width, float height)
{
    return new Vector2(
        Main.rand.NextFloat(-width / 2f, width / 2f),
        Main.rand.NextFloat(-height / 2f, height / 2f)
    );
}

// Circulo solido: usar coordenadas polares
public static Vector2 RandomPointInCircle(float radius)
{
    float distance = Main.rand.NextFloat(0f, radius);
    float radians = Main.rand.NextFloat(0f, MathHelper.TwoPi);
    return new Vector2(distance, 0f).RotatedBy(radians);
}

// Box hueca: elegir un borde aleatorio y posicion en ese borde
public static Vector2 RandomPointOnHollowBox(float width, float height)
{
    int edge = Main.rand.Next(4);
    return edge switch
    {
        0 => new Vector2(Main.rand.NextFloat(-width/2, width/2), -height/2), //
top
        1 => new Vector2(width/2, Main.rand.NextFloat(-height/2, height/2)), //
right
        2 => new Vector2(Main.rand.NextFloat(-width/2, width/2), height/2), //
bottom
        _ => new Vector2(-width/2, Main.rand.NextFloat(-height/2, height/2)), //
left
    };
}

// Circulo hueco: angulo aleatorio, radio fijo
public static Vector2 RandomPointOnHollowCircle(float radius)
{
    float radians = Main.rand.NextFloat(0f, MathHelper.TwoPi);
    return new Vector2(radius, 0f).RotatedBy(radians);
}
```

5.5 Sistemas de coordenadas: world vs screen

Terraria usa dos sistemas de coordenadas: coordenadas del mundo (world coordinates, donde el origen es la esquina superior izquierda del mundo) y coordenadas de pantalla (screen coordinates, donde el origen es la esquina superior izquierda de la ventana). Para renderizar una partícula, se resta `Main.screenPosition` a la posición del mundo para obtener la posición en pantalla. Es crucial entender esto porque muchos bugs visuales provienen de mezclar ambos sistemas.

```
// Posicion en mundo (donde vive la partícula logicamente)
Vector2 worldPos = new Vector2(1000f, 500f);

// Posicion en pantalla (donde se dibuja)
Vector2 screenPos = worldPos - Main.screenPosition;

// En el metodo Draw de la partícula:
spriteBatch.Draw(texture, worldPos - Main.screenPosition, color);
```

5.6 Time y framerate independence

Las partículas se actualizan en `Update()` que se llama una vez por frame. La duración de un frame varía: a 60 FPS son 16.67ms, a 144 FPS son 6.94ms. Si la lógica de la partícula usa velocidad por frame (`position += velocity`), el resultado será distinto según el framerate. Para ser framerate-independent, se debe multiplicar por $(\text{deltaTime} / \text{fixedStep})$. Sin embargo, Terraria usa un fixed timestep de 60Hz por defecto, así que en la práctica las partículas se actualizan a 60Hz sin importar el framerate del display. Esto simplifica el código: se puede asumir que cada `Update` es 1/60 de segundo.

6. MonoGame y XNA Graphics Pipeline

6.1 Que es MonoGame

MonoGame es un framework open-source para hacer juegos en C# que evolucionó desde Microsoft XNA. Terraria está construido sobre XNA 4.0 (versión original) y tModLoader 1.4.4 usa FNA (reimplementación open-source de XNA 4.0, API compatible con MonoGame) para compatibilidad cross-platform. La API de MonoGame es casi idéntica a XNA, así que cualquier referencia a "XNA" aplica también a MonoGame. Entender el pipeline de rendering de MonoGame es esencial para implementar partículas eficientemente.

6.2 SpriteBatch: el caballo de batalla

`SpriteBatch` es la API principal de MonoGame para dibujar texturas 2D. Agrupa múltiples draw calls en un solo batch para reducir overhead. En Terraria, `Main.spriteBatch` es la instancia global que se usa para todo el rendering 2D. Para dibujar partículas, se usa `spriteBatch.Draw()` con varias sobrecargas que aceptan textura, posición, rectángulo fuente, color, rotación, origen, escala, efectos y layer depth.

```
// Draw basico
spriteBatch.Draw(texture, position, color);

// Draw con rectangulo fuente (para spritesheets)
spriteBatch.Draw(texture, position, new Rectangle(0, 0, 32, 32), color);
```

```
// Draw completo con rotacion, origen, escala
spriteBatch.Draw(
    texture,                // Texture2D
    position,               // Vector2 (screen coords)
    sourceRect,             // Rectangle? (null = textura completa)
    color,                  // Color (tint + alpha)
    rotation,               // float (radianes)
    origin,                 // Vector2 (punto de rotacion/escala)
    scale,                  // Vector2 o float
    effects,                // SpriteEffects (FlipHorizontally, etc.)
    layerDepth              // float (0=frente, 1=fondo)
);
```

6.3 Begin y End: blend modes y save state

Antes de dibujar, se llama `spriteBatch.Begin()` con parametros que controlan blend mode, sampler state, depth stencil state, rasterizer state, y transform matrix. Después de dibujar todas las partículas, se llama `spriteBatch.End()`. Es importante entender los blend modes:

BlendState	Formula	Uso tipico
AlphaBlend	$dest = src * src.A + dest * (1 - src.A)$	Sprites normales con transparencia
Additive	$dest = src * src.A + dest$	Glow, fuego, efectos magicos, lightning
NonPremultiplied	$dest = src + dest * (1 - src.A)$	Sprites sin premultiplied alpha
Opaque	$dest = src$	Sin blending (reemplaza)

Para partículas, Additive es muy usado para efectos de glow (fuego, magia, lightning) porque suma colores creando areas mas brillantes donde se solapan. AlphaBlend es estandar para humo, polvo y partículas solidas. Cambiar blend state entre Begin/End es costoso, así que se agrupan partículas por blend mode.

6.4 El pipeline de rendering de Terraria

Terraria renderiza su mundo en un orden especifico. Los hooks de `tModLoader` se inyectan en puntos especificos de este pipeline. El orden aproximado es: clear backbuffer, dibujar fondo lejano, dibujar background walls, dibujar tiles no solidos, dibujar NPCs detras de tiles, dibujar tiles solidos, dibujar NPCs, dibujar projectiles, dibujar players, dibujar items dropped, dibujar rain, dibujar gore, dibujar dusts, dibujar interface, dibujar mouse cursor. Para inyectar partículas en un punto especifico, se usa el hook correspondiente via `ModSystem`.

6.5 Shaders en MonoGame

MonoGame soporta shaders HLSL (High-Level Shading Language) compilados a formato `.xnb`. Hay dos tipos de shaders relevantes para partículas: pixel shaders (que manipulan el color de cada pixel) y vertex shaders (que manipulan la posicion y atributos de vertices). Para partículas con efectos custom (distorsion, color shifts procedural, etc.), se usan shaders. Para rendering estandar, `spriteBatch.Draw()` con blend modes suele ser suficiente.

Los shaders se cargan como Effect objects en MonoGame. Para aplicar un shader durante el drawing, se llama `spriteBatch.Begin(SpriteSortMode.Immediate, blendState, ..., effect)` o se usa `CurrentTechnique.Passes[0].Apply()` dentro de un Begin/End. Es crucial entender que cambiar shaders tiene overhead, así que se agrupan partículas que usan el mismo shader.

7. El Sistema Dust Vanilla de Terraria

7.1 Que es Dust y por que existe

Dust es el sistema de partículas nativo de Terraria. Está implementado en la clase `Terraria.Dust` y consiste en un array pre-asignado de 6000 partículas (`Main.dust[6000]`) que se reciclan. Cuando se llama `Dust.NewDust()`, se busca un slot libre y se inicializa. Cuando una partícula expira (`timeLeft` llega a 0 o `active = false`), el slot se marca como libre para reuso. Este diseño evita allocations de GC durante el juego, lo cual es crítico para mantener 60 FPS estables.

El sistema Dust está optimizado para el caso de uso de Terraria: cientos de partículas activas simultáneamente, lifetime corto (típicamente 30-300 ticks), comportamiento simple (movimiento lineal con gravedad opcional, fade out, sin rotación compleja). Para casos que se ajustan a este patrón, Dust es suficiente. El problema aparece cuando se necesitan features que Dust no soporta bien: rotación real (`Dust.rotation` solo afecta el dibujo, no la dirección de movimiento), scale asimétrica (`Dust.scale` es un float, no `Vector2`), colores que cambian durante la vida (`Dust.color` es estático), texturas custom (limitado a las 300+ vanilla), y blending modes por partícula (Dust usa el blend mode global del `SpriteBatch`).

7.2 API de Dust

```
// Spawnear una partícula de polvo basica
int dustId = Dust.NewDust(position, width, height, dustType, velocityX, velocityY,
                           alpha, color, scale);

// Configurar la partícula despues de spawnear
Dust dust = Main.dust[dustId];
dust.noGravity = true;
dust.noLight = true;
dust.velocity *= 0.95f; // friccion
dust.scale = 1.5f;
dust.fadeIn = 0.5f;
dust.rotation = 0f;
dust.color = Color.Red;

// Dust custom (ModDust)
// Requiere registrarlo en Mod.Load():
// AddContent(new MiDustCustom());
public class MiDustCustom : ModDust
{
    public override void OnSpawn(Dust dust) {
        dust.noGravity = true;
        dust.frame = new Rectangle(0, 0, 8, 8);
    }

    public override bool Update(Dust dust) {
        dust.position += dust.velocity;
        dust.velocity *= 0.98f;
        dust.scale -= 0.01f;
        if (dust.scale < 0.1f) dust.active = false;
        return false; // false = no aplicar fisica vanilla
    }
}
```

7.3 Limitaciones criticas de Dust

Limitacion	Impacto	Solucion en libreria propia
Solo 6000 slots	En boss fights complejos se saturan	Buffer configurable hasta 100k+
Sin rotacion real	Las partículas no pueden "apuntar" a su direccion	Rotation field real
Scale unidimensional	No se puede estirar en un eje	Scale como Vector2
Color estatico	No se puede interpolar color durante vida	StartColor + EndColor + lerp
Texturas limitadas	Solo IDs vanilla o ModDust custom (caro)	TextureId ushort, 65k texturas
Sin blending por partícula	Todo usa el blend mode global del SpriteBatch	Batching por blend mode
Sin components	Cada tipo de partícula necesita ModDust propio	Sistema component-based
Sin layers custom	Solo se renderiza en un punto fijo del pipeline	LayerPriority numerica

7.4 Por que no usar Dust directamente

Para mods simples que solo necesitan chispas o polvo basico, Dust vanilla es perfecto: es rapido, esta bien optimizado, y no requiere dependencias externas. Sin embargo, para mods que quieren VFX sofisticados (slash effects estilo Calamity, auras orbitales, partículas con trail, efectos magicos complejos), Dust se queda corto. Una libreria de particulas propia resuelve estas limitaciones pagando un costo en complejidad y en bytes adicionales de memoria.

8. Por que Dust no es Suficiente: Casos de Uso Avanzados

8.1 Slash effects estilo Calamity

Un slash effect es una estela curva visible que sigue la trayectoria de un arma melee al golpear. Visualmente es dramatico y es la marca de la casa de Calamity. Implementarlo con Dust es practicamente imposible por varias razones: la partícula necesita rotar para orientarse segun la direccion del swing, necesita un trail continuo (no partículas discretas), necesita blending aditivo para brillar, y necesita scale asimetrica para formar el crescent. Una libreria propia resuelve esto con un ModProjectile custom con PreDraw que dibuja un crescent texturizado con rotacion y scale.

8.2 Auras orbitales

Un aura orbital es un conjunto de partículas que orbitan al jugador formando un patron circular. Es tipico de accesorios "magicos" o buffs de poder. Con Dust, mantener la posicion orbital requiere recalculer cada frame la posicion angular de cada partícula, y como Dust no tiene estado custom, esto requiere un ModProjectile que trackee las partículas. Una libreria propia puede ofrecer un componente OrbitComponent que se aplica a N partículas y gestiona la rotacion automaticamente.

8.3 Trails continuos

Un trail continuo es una estela detras de un proyectil en movimiento. Visualmente es lo que diferencia un proyectil basico de uno "chulo". Con Dust, se puede spawnear una partícula por frame en la posicion del proyectil, pero el resultado es discreto (puntos separados), no continuo. Una libreria propia puede usar el array oldPos[] del proyectil y dibujar un trail degradado continuo en PreDraw, mucho mas eficiente visualmente.

8.4 Particulas con homing

Homing significa que la partícula persigue un target (NPC o jugador). Es util para proyectiles magicos "inteligentes" o para efectos de healing. Con Dust, implementar homing requiere overridear Update en ModDust y buscar el NPC mas cercano cada frame, lo cual es costoso. Una libreria propia puede tener un HomingComponent con un target pre-asignado (no hay que buscar cada frame) y logica optimizada de interpolacion de velocidad.

8.5 Screenshake coordinado

Aunque estrictamente no es un feature de partículas, el screenshake coordina con VFX para crear impacto. Terraria ofrece PunchCameraModifier que es simple de usar, pero coordinarlo con la explosion de partículas requiere hooks globales. Una libreria propia puede exponer una API que spawnear partículas y aplica screenshake en una sola llamada.

9. Diseno Data-Oriented vs Object-Oriented

9.1 Que es data-oriented design (DOD)

Data-oriented design es un paradigma de programacion que prioriza la organizacion de los datos para acceso eficiente, en contraste con object-oriented programming (OOP) que prioriza la organizacion del codigo por conceptos. En DOD, los datos del mismo tipo se almacenan en arrays contiguos (Structure of Arrays, SoA) en lugar de arrays de objetos (Array of Structures, AoS). Esto mejora cache locality porque cuando se accede a la posicion de una partícula, las posiciones de las siguientes tambien estan en cache.

Para una libreria de partículas que maneja miles de instancias, DOD ofrece 2-5x mejor rendimiento que OOP en operaciones de iteracion. La desventaja es que la API es menos intuitiva: en lugar de

particle.Update(), se tiene ParticleSystem.Update(particles, count). La solución es ofrecer una API de alto nivel que internamente use DOD.

9.2 SoA vs AoS

Array of Structures (AoS) almacena cada partícula como un struct completo: [P1(pos,vel,col), P2(pos,vel,col), P3(pos,vel,col)]. Cuando iteras accediendo solo a posición, el cache trae también velocidad y color innecesariamente. Structure of Arrays (SoA) almacena cada campo por separado: [pos1,pos2,pos3], [vel1,vel2,vel3], [col1,col2,col3]. Iterar sobre posiciones solo trae posiciones al cache.

```
// AoS - clasico OOP
public struct Particle {
    public Vector2 Position;
    public Vector2 Velocity;
    public Color Color;
}
Particle[] particles = new Particle[10000];
// Iterar trae todo al cache, desperdiciando ancho de banda
for (int i = 0; i < particles.Length; i++)
    particles[i].Position += particles[i].Velocity;

// SoA - data-oriented
public class ParticleSystem {
    public Vector2[] Positions = new Vector2[10000];
    public Vector2[] Velocities = new Vector2[10000];
    public Color[] Colors = new Color[10000];
}
// Iterar solo trae Positions y Velocities al cache
for (int i = 0; i < system.Positions.Length; i++)
    system.Positions[i] += system.Velocities[i];
```

9.3 Compromiso: struct con fields relevantes

El enfoque puro SoA es difícil de mantener en código porque pierde la coherencia conceptual (no hay un tipo "Particle" como tal). Un compromiso común es usar un struct ParticleData con todos los campos, agrupado en un array, y aceptar que el cache trae algo de información extra. Si el struct es pequeño (menos de 64 bytes), el impacto es mínimo. Esta es la approach que adopta esta librería.

10. Structs vs Clases para Partículas

10.1 Diferencias técnicas en C#

En C#, las structs son value types: se almacenan donde se declaran (en el stack si son locales, en el objeto contenedor si son fields). Las clases son reference types: siempre se almacenan en el heap, y la variable es una referencia. Para arrays, un array de structs contiene los structs directamente en memoria contigua, mientras que un array de clases contiene referencias (punteros) a objetos dispersos en el heap.

Aspecto	Struct	Class
Almacenamiento	Donde se declara	Heap

Aspecto	Struct	Class
Array	Contiguo en memoria	Array de punteros
Cache locality	Excelente	Pobre (pointer chasing)
Allocation	Sin GC (en stack/contenedor)	GC presiona
Herencia	No soportada	Soportada
Polimorfismo	Solo via interfaces	Virtual methods
Tamaño	Fijo (suma de fields)	Overhead de object header
Null	No (es valor)	Si
Copia en asignacion	Copia valores	Copia referencia

10.2 Diseno del struct ParticleData

```
using Microsoft.Xna.Framework;
using System.Runtime.InteropServices;

namespace MiParticleLib.Core
{
    /// <summary>
    /// Representa una partícula individual. Es un struct para maximizar
    /// cache locality cuando se itera sobre miles de instancias.
    /// </summary>
    [StructLayout(LayoutKind.Sequential, Pack = 1)]
    public struct ParticleData
    {
        // Posicion en mundo (8 bytes)
        public Vector2 Position;

        // Velocidad por frame (8 bytes)
        public Vector2 Velocity;

        // Escala X/Y separadas para soportar estiramiento (8 bytes)
        public Vector2 Scale;

        // Color packed como uint (RGBA, 4 bytes)
        public uint PackedColor;

        // Color inicial (para lerp durante vida) (4 bytes)
        public uint PackedStartColor;

        // Color final (4 bytes)
        public uint PackedEndColor;

        // Rotacion en radianes (4 bytes)
        public float Rotation;

        // Velocidad angular (4 bytes)
        public float RotationSpeed;

        // Tiempo restante de vida en ticks (4 bytes)
        public int TimeLeft;

        // Duracion total para calcular progress 0-1 (4 bytes)
        public int Duration;

        // ID de textura (lookup en tabla) (2 bytes)
        public ushort TextureId;

        // Layer priority para orden de render (2 bytes)
        public ushort LayerPriority;

        // Bitmask de componentes activos (8 bytes)
        public ulong ComponentFlags;

        // Blend mode (0=Alpha, 1=Additive, 2=NonPremultiplied) (1 byte)
        public byte BlendMode;

        // Flags miscelaneos (1 byte)
        public byte Flags; // bit 0: active, bit 1: dirty

        // Datos custom para componentes (16 bytes, 4 floats)
        public float UserData0;
        public float UserData1;
        public float UserData2;
        public float UserData3;
    }
}
```

```

        // Total: ~82 bytes. Excede una linea de cache tipica (64 bytes), pero los
campos mas

        // Helper: esta activa?
public bool IsActive => (Flags & 0x01) != 0;

        // Helper: progress de vida (1.0 = recién nacida, 0.0 = muerta)
public float LifeProgress => Duration > 0 ? TimeLeft / (float)Duration :
0f;

        // Helper: activar/desactivar
public void SetActive(bool active)
{
    if (active) Flags |= 0x01;
    else Flags &= 0xFE;
}

        // Helper: tiene componente activo
public bool HasComponent(ulong flag) => (ComponentFlags & flag) != 0;

        // Helper: activar componente
public void EnableComponent(ulong flag) => ComponentFlags |= flag;

        // Helper: desactivar componente
public void DisableComponent(ulong flag) => ComponentFlags &= ~flag;
    }
}

```

10.3 Component flags

El campo `ComponentFlags` es un `ulong` (64 bits) que actúa como bitmask para indicar que componentes están activos en esta partícula. Cada componente tiene un bit asignado. Cuando `ParticleProcessor` itera sobre todas las partículas, para cada una chequea que componentes tiene activos y ejecuta la lógica correspondiente. Esto es extremadamente rápido porque es una operación AND a nivel de bits.

```

public static class ComponentFlag
{
    public const ulong Gravity          = 1UL << 0;    // bit 0
    public const ulong FadeOut          = 1UL << 1;    // bit 1
    public const ulong FadeIn           = 1UL << 2;    // bit 2
    public const ulong ScaleDown        = 1UL << 3;    // bit 3
    public const ulong ScaleUp          = 1UL << 4;    // bit 4
    public const ulong Homing           = 1UL << 5;    // bit 5
    public const ulong Rotation         = 1UL << 6;    // bit 6
    public const ulong ColorShift       = 1UL << 7;    // bit 7
    public const ulong Orbit            = 1UL << 8;    // bit 8
    public const ulong Trail            = 1UL << 9;    // bit 9
    public const ulong BounceOnTile     = 1UL << 10;   // bit 10
    public const ulong DieOnTile        = 1UL << 11;   // bit 11
    public const ulong EmitLight        = 1UL << 12;   // bit 12
    public const ulong Custom1          = 1UL << 13;   // bit 13 (usuario)
    public const ulong Custom2          = 1UL << 14;   // bit 14 (usuario)
    public const ulong Custom3          = 1UL << 15;   // bit 15 (usuario)
    // ... hasta 64 bits
}

```

11. Buffers Pre-asignados y Gestion de Memoria

11.1 Por que pre-asignar

En C#, asignar objetos en el heap genera presion sobre el Garbage Collector (GC). Cuando el GC se ejecuta, pausa todos los threads para compactar memoria, lo cual causa "hitches" (caidas momentaneas de framerate). Para un juego que corre a 60 FPS y spawna/despawnea cientos de partículas por frame, asignar y liberar objetos constantemente causaria hitches visibles. La solucion es pre-asignar un buffer de N partículas al inicio y reciclar slots.

11.2 Implementacion del buffer

NOTA IMPORTANTE sobre structs en C#: los structs son value types. Cuando accedes a array[i] (siendo array de structs), el indexer devuelve una COPIA, no una referencia. Llamar array[i].MetodoMutador() opera sobre esa copia temporal que se descarta. Para modificar el elemento del array se debe usar la palabra clave ref (ref ParticleData p = ref array[i]; p.MetodoMutador();) o re-asignar el struct completo (var p = data; p.Mutate(); array[i] = p;). Ademas, Action<T1, T2> NO soporta parametros ref, por lo que se define un delegate custom RefParticleAction. Estas dos particularidades son fundamentales para que el codigo del buffer compile y funcione correctamente.

```
// Delegate custom necesario porque Action<,> no soporta ref params
public delegate void RefParticleAction(int index, ref ParticleData data);

public class ParticleBuffer
{
    private readonly ParticleData[] _particles;
    private readonly int _capacity;
    private int _activeCount;
    private int _nextSlot; // Round-robin para buscar slots libres

    public int Capacity => _capacity;
    public int ActiveCount => _activeCount;
    public ParticleData[] RawData => _particles;

    public ParticleBuffer(int capacity)
    {
        _capacity = capacity;
        _particles = new ParticleData[capacity];
        _activeCount = 0;
        _nextSlot = 0;
    }

    /// <summary>
    /// Intenta spawnear una nueva partícula. Devuelve false si el buffer esta
    /// lleno.
    /// </summary>
    public bool TrySpawn(ParticleData data)
    {
        // Buscar slot libre empezando en _nextSlot
        for (int i = 0; i < _capacity; i++)
        {
            int idx = (_nextSlot + i) % _capacity;
            if (!_particles[idx].IsActive)
```



```

        {
            _particles[idx] = data;
            // Nota: activamos el flag antes de asignar. Hacerlo despues
            // (_particles[idx].SetActive(true)) NO funciona porque structs
son
            // value types y array[i] devuelve una copia temporal descartable.
            data.SetActive(true); // alternativamente: ya viene activado
            _particles[idx] = data; // re-asignar con flag activado
            _nextSlot = (idx + 1) % _capacity;
            _activeCount++;
            return true;
        }
    }
    return false; // Buffer lleno
}

/// <summary>
/// Itera sobre todas las partículas activas y aplica una accion.
/// </summary>
public void ForEachActive(RefParticleAction action)
{
    for (int i = 0; i < _capacity; i++)
    {
        if (_particles[i].IsActive)
        {
            action(i, ref _particles[i]);
        }
    }
}

/// <summary>
/// Desactiva una partícula por indice.
/// </summary>
public void Kill(int index)
{
    if (index >= 0 && index < _capacity && _particles[index].IsActive)
    {
        // Usar ref para modificar el struct directamente en el array
        ref ParticleData p = ref _particles[index];
        p.SetActive(false);
        _activeCount--;
    }
}

/// <summary>
/// Desactiva todas las partículas.
/// </summary>
public void Clear()
{
    for (int i = 0; i < _capacity; i++)
    {
        ref ParticleData p = ref _particles[i];
        p.SetActive(false);
    }
    _activeCount = 0;
    _nextSlot = 0;
}
}

```

11.3 Estrategia round-robin

La búsqueda de slot libre usa round-robin: en lugar de empezar siempre desde 0, se empieza desde el ultimo slot asignado. Esto distribuye la carga y evita que las primeras posiciones del array se desgasten mas que las ultimas. En promedio, la búsqueda es $O(1)$ amortizado cuando hay slots libres, lo cual es critico para mantener performance.

11.4 Capacidad configurable

El tamaño del buffer se configura en OnModLoad basado en ModConfig. Para hardware modesto, 2000 partículas puede ser suficiente. Para hardware potente con mods VFX intensivos, 10000 es razonable. Es importante no excederse porque cada partícula ocupa ~80 bytes, así que 100k partículas son 8 MB solo del buffer, mas overhead de rendering.

12. Sistema de Components

12.1 Diseno de IComponent

Un componente es una unidad de comportamiento que se puede adjuntar a una partícula. Por ejemplo: GravityComponent aplica gravedad a la velocidad, FadeOutComponent reduce alpha con el tiempo, HomingComponent persigue un target. La ventaja del enfoque component-based es que se pueden combinar libremente: una partícula puede tener Gravity + FadeOut + Homing sin necesidad de crear una clase nueva.

```
public interface IParticleComponent
{
    /// <summary>Bit flag unico para este componente.</summary>
    ulong Flag { get; }

    /// <summary>
    /// Se llama cada frame para todas las partículas que tienen este componente
    activo.
    /// </summary>
    void Update(ref ParticleData particle, int index);

    /// <summary>
    /// Se llama al activar el componente en una partícula (para inicializar
    UserData).
    /// </summary>
    void OnEnable(ref ParticleData particle);
}
```

12.2 Componentes implementados

```
public class GravityComponent : IParticleComponent
{
    public ulong Flag => ComponentFlag.Gravity;
    public float Gravity { get; set; } = 0.2f;

    public void OnEnable(ref ParticleData particle)
    {
        // UserData0 almacena la gravedad custom (si 0, usar default)
        if (particle.UserData0 == 0) particle.UserData0 = Gravity;
    }

    public void Update(ref ParticleData particle, int index)
    {
        particle.Velocity.Y += particle.UserData0;
    }
}

public class FadeOutComponent : IParticleComponent
{
    public ulong Flag => ComponentFlag.FadeOut;
    public float FadeStart { get; set; } = 0.5f; // Empieza a fade al 50% de vida

    public void OnEnable(ref ParticleData particle) { }

    public void Update(ref ParticleData particle, int index)
    {
        float progress = particle.LifeProgress; // 1.0 -> 0.0
        if (progress < FadeStart)
        {
            float fadeAmount = progress / FadeStart; // 1 -> 0
            // Aplicar alpha al color packed
            byte currentAlpha = (byte)(particle.PackedColor >> 24);
            byte newAlpha = (byte)(currentAlpha * fadeAmount);
            particle.PackedColor = (particle.PackedColor & 0x00FFFFFF) |
            ((uint)newAlpha << 24);
        }
    }
}

public class HomingComponent : IParticleComponent
{
    public ulong Flag => ComponentFlag.Homing;
    public float HomingStrength { get; set; } = 0.1f;

    public void OnEnable(ref ParticleData particle) { }

    public void Update(ref ParticleData particle, int index)
    {
        // UserData0 almacena el whoAmI del target NPC
        int targetWhoAmI = (int)particle.UserData0;
        if (targetWhoAmI < 0 || targetWhoAmI >= Main.npc.Length) return;

        NPC target = Main.npc[targetWhoAmI];
        if (!target.active) return;

        Vector2 toTarget = target.Center - particle.Position;
        if (toTarget.LengthSquared() < 1f) return;
        toTarget.Normalize();

        particle.Velocity = Vector2.Lerp(
            particle.Velocity, toTarget * particle.Velocity.Length(),
            HomingStrength);
    }
}
```

```

    }
}

public class ScaleAnimationComponent : IParticleComponent
{
    public ulong Flag => ComponentFlag.ScaleDown;
    public Vector2 StartScale { get; set; } = Vector2.One;
    public Vector2 EndScale { get; set; } = Vector2.Zero;

    public void OnEnable(ref ParticleData particle)
    {
        particle.Scale = StartScale;
    }

    public void Update(ref ParticleData particle, int index)
    {
        float t = 1f - particle.LifeProgress; // 0 -> 1
        particle.Scale = Vector2.Lerp(StartScale, EndScale, t);
    }
}

public class ColorShiftComponent : IParticleComponent
{
    public ulong Flag => ComponentFlag.ColorShift;

    public void OnEnable(ref ParticleData particle) { }

    public void Update(ref ParticleData particle, int index)
    {
        float t = 1f - particle.LifeProgress; // 0 -> 1
        // Interpolare entre StartColor y EndColor
        Color start = UnpackColor(particle.PackedStartColor);
        Color end = UnpackColor(particle.PackedEndColor);
        Color current = Color.Lerp(start, end, t);
        particle.PackedColor = PackColor(current);
    }

    private static Color UnpackColor(uint packed) =>
        new Color(
            (byte)(packed & 0xFF),
            (byte)((packed >> 8) & 0xFF),
            (byte)((packed >> 16) & 0xFF),
            (byte)((packed >> 24) & 0xFF)
        );

    private static uint PackColor(Color c) =>
        (uint)c.R | ((uint)c.G << 8) | ((uint)c.B << 16) | ((uint)c.A << 24);
}

```

13. Sistema de Emitters

13.1 Que es un emitter

Un emitter es un objeto que genera partículas en una posición con una forma geométrica. Por ejemplo: un emitter "circular" genera partículas dentro de un círculo alrededor de su posición; un emitter "cono" genera partículas en un cono apuntando a una dirección. Los emitters encapsulan la lógica de "donde y como spawnear", dejando al usuario solo especificar "que" y "cuando". Esto simplifica enormemente la creación de efectos complejos.

13.2 Clase base Emitter

```
public abstract class Emitter
{
    public Vector2 Position { get; set; }
    public ShapeDescriptor Shape { get; set; }
    public int SpawnRate { get; set; } = 1; // Partículas por frame
    public bool IsActive { get; protected set; } = true;

    protected Emitter(Vector2 position, ShapeDescriptor shape)
    {
        Position = position;
        Shape = shape;
    }

    /// <summary>Se llama cada frame. Override para implementar logica
    custom.</summary>
    public virtual void Update()
    {
        if (!IsActive) return;

        for (int i = 0; i < SpawnRate; i++)
        {
            SpawnParticle();
        }
    }

    /// <summary>Genera una partícula con posicion segun la shape. Override para
    customizar.</summary>
    protected virtual void SpawnParticle()
    {
        Vector2 offset = Shape.GenerateRandomPoint();
        Vector2 spawnPos = Position + offset;

        var data = new ParticleData
        {
            Position = spawnPos,
            Velocity = Vector2.Zero,
            Scale = Vector2.One,
            TimeLeft = 60,
            Duration = 60,
            TextureId = 0, // Default texture
        };

        OnSpawnParticle(ref data);
        ParticleManager.Spawn(data);
    }

    /// <summary>Hook para que subclasses configuren la partícula antes de
    spawnear.</summary>
    protected virtual void OnSpawnParticle(ref ParticleData data) { }
}
```

13.3 Implementacion de ShapeDescriptor

```
public readonly struct ShapeDescriptor
{
    public ShapeType Type { get; }
    public Vector2 Size { get; }
    public bool Hollow { get; }
    public float Rotation { get; }

    public ShapeDescriptor(ShapeType type, Vector2 size, bool hollow, float rotation)
    {
        Type = type; Size = size; Hollow = hollow; Rotation = rotation;
    }

    public Vector2 GenerateRandomPoint()
    {
        Vector2 point = Type switch
        {
            ShapeType.Box => GenerateBox(Size.X, Size.Y, Hollow),
            ShapeType.Circle => GenerateCircle(Size.X, Hollow),
            ShapeType.Cone => GenerateCone(Size.X, Size.Y),
            ShapeType.Sphere => GenerateSphere(Size.X),
            ShapeType.Vortex => GenerateVortex(Size.X, Size.Y),
            ShapeType.Line => GenerateLine(Size.X),
            _ => Vector2.Zero,
        };

        // Aplicar rotacion si no es cero
        if (Rotation != 0f)
            point = point.RotatedBy(Rotation);

        return point;
    }

    private static Vector2 GenerateBox(float w, float h, bool hollow)
    {
        if (hollow)
        {
            int edge = Main.rand.Next(4);
            return edge switch
            {
                0 => new Vector2(Main.rand.NextFloat(-w/2, w/2), -h/2),
                1 => new Vector2(w/2, Main.rand.NextFloat(-h/2, h/2)),
                2 => new Vector2(Main.rand.NextFloat(-w/2, w/2), h/2),
                _ => new Vector2(-w/2, Main.rand.NextFloat(-h/2, h/2)),
            };
        }
        return new Vector2(
            Main.rand.NextFloat(-w/2, w/2),
            Main.rand.NextFloat(-h/2, h/2)
        );
    }

    private static Vector2 GenerateCircle(float r, bool hollow)
    {
        float angle = Main.rand.NextFloat(0f, MathHelper.TwoPi);
        float dist = hollow ? r : Main.rand.NextFloat(0f, r);
        return new Vector2(dist, 0f).RotatedBy(angle);
    }

    private static Vector2 GenerateCone(float length, float halfAngle)
    {
        float angle = Main.rand.NextFloat(-halfAngle, halfAngle);
```

```

        float dist = Main.rand.NextFloat(0f, length);
        return new Vector2(dist, 0f).RotatedBy(angle);
    }

    private static Vector2 GenerateSphere(float r)
    {
        // Aproximacion 2D de esfera 3D: distribucion no uniforme
        float angle = Main.rand.NextFloat(0f, MathHelper.TwoPi);
        // sqrt para distribucion uniforme en area
        float dist = (float)Math.Sqrt(Main.rand.NextFloat(0f, 1f)) * r;
        return new Vector2(dist, 0f).RotatedBy(angle);
    }

    private static Vector2 GenerateVortex(float r, float turns)
    {
        float t = Main.rand.NextFloat(0f, 1f);
        float angle = t * turns * MathHelper.TwoPi;
        float dist = t * r;
        return new Vector2(dist, 0f).RotatedBy(angle);
    }

    private static Vector2 GenerateLine(float length)
    {
        return new Vector2(Main.rand.NextFloat(-length/2, length/2), 0f);
    }

    // Factory methods
    public static ShapeDescriptor Box(float w, float h) =>
        new(ShapeType.Box, new Vector2(w, h), false, 0f);
    public static ShapeDescriptor Circle(float r) =>
        new(ShapeType.Circle, new Vector2(r, r), false, 0f);
    public static ShapeDescriptor HollowBox(float w, float h) =>
        new(ShapeType.Box, new Vector2(w, h), true, 0f);
    public static ShapeDescriptor HollowCircle(float r) =>
        new(ShapeType.Circle, new Vector2(r, r), true, 0f);
    public static ShapeDescriptor Cone(float length, float halfAngle) =>
        new(ShapeType.Cone, new Vector2(length, halfAngle), false, 0f);
    public static ShapeDescriptor Sphere(float r) =>
        new(ShapeType.Sphere, new Vector2(r, r), false, 0f);
    public static ShapeDescriptor Vortex(float r, float turns) =>
        new(ShapeType.Vortex, new Vector2(r, turns), false, 0f);
    public static ShapeDescriptor Line(float length) =>
        new(ShapeType.Line, new Vector2(length, 0), false, 0f);
}

```

14. ParticleManager: Orquestador Central

14.1 Responsabilidades del manager

ParticleManager es el singleton que coordina todo el sistema. Sus responsabilidades son: mantener el buffer principal, registrar components, exponer la API publica de spawn, actualizar todas las partículas cada frame, y renderizarlas en el orden correcto. Es la unica clase que el usuario final invoca directamente en la mayoria de casos.

```

public static class ParticleManager
{
    private static ParticleBuffer _buffer;
    private static Dictionary<ulong, IParticleComponent> _components;

```

```

private static Texture2D[] _textures;
private static Dictionary<string, ushort> _textureLookup;

public static int Capacity => _buffer?.Capacity ?? 0;
public static int ActiveCount => _buffer?.ActiveCount ?? 0;

public static void Initialize(int capacity = 5000)
{
    _buffer = new ParticleBuffer(capacity);
    _components = new Dictionary<ulong, IParticleComponent>();
    _textures = new Texture2D[256];
    _textureLookup = new Dictionary<string, ushort>();

    // Registrar components built-in
    RegisterComponent(new GravityComponent());
    RegisterComponent(new FadeOutComponent());
    RegisterComponent(new FadeInComponent());
    RegisterComponent(new ScaleAnimationComponent());
    RegisterComponent(new HomingComponent());
    RegisterComponent(new ColorShiftComponent());
    RegisterComponent(new RotationComponent());
    RegisterComponent(new OrbitComponent());
}

public static void Shutdown()
{
    _buffer?.Clear();
    _buffer = null;
    _components?.Clear();
    _components = null;
    _textures = null;
    _textureLookup?.Clear();
    _textureLookup = null;
}

public static void RegisterComponent(IParticleComponent component)
{
    _components[component.Flag] = component;
}

public static ushort RegisterTexture(string path)
{
    if (_textureLookup.TryGetValue(path, out ushort id))
        return id;

    // Buscar slot libre
    for (ushort i = 0; i < _textures.Length; i++)
    {
        if (_textures[i] == null)
        {
            _textures[i] = ModContent.Request<Texture2D>(path).Value;
            _textureLookup[path] = i;
            return i;
        }
    }
    return 0; // Fallback a default
}

/// <summary>API publica principal: spawnear una partícula.</summary>
public static bool Spawn(ParticleData data)
{
    return _buffer?.TrySpawn(data) ?? false;
}

```



```

    /// <summary>API conveniente: spawnear N partículas en una shape.</summary>
    public static void SpawnShape(Vector2 center, ShapeDescriptor shape, int
count,
                                ParticleData template)
    {
        for (int i = 0; i < count; i++)
        {
            Vector2 offset = shape.GenerateRandomPoint();
            ParticleData p = template;
            p.Position = center + offset;
            Spawn(p);
        }
    }

    /// <summary>Update de todas las partículas. Llamado por
ModSystem.PostUpdateWorld.</summary>
    public static void Update()
    {
        if (_buffer == null) return;

        _buffer.ForEachActive((index, ref p) =>
        {
            // Aplicar components
            foreach (var kvp in _components)
            {
                if (p.HasComponent(kvp.Key))
                {
                    kvp.Value.Update(ref p, index);
                }
            }

            // Movimiento base (siempre se aplica)
            p.Position += p.Velocity;
            p.Rotation += p.RotationSpeed;

            // Decrementar tiempo de vida
            p.TimeLeft--;
            if (p.TimeLeft <= 0)
            {
                p.SetActive(false);
            }
        }
    });

    /// <summary>Render de todas las partículas. Llamado por ModSystem en hooks de
draw.</summary>
    public static void Render(SpriteBatch spriteBatch, CameraBounds bounds)
    {
        if (_buffer == null) return;

        // Agrupar por blend mode para minimizar state changes
        // Render en 3 pasadas: Alpha, Additive, NonPremultiplied
        RenderWithBlend(spriteBatch, bounds, BlendState.AlphaBlend, 0);
        RenderWithBlend(spriteBatch, bounds, BlendState.Additive, 1);
        RenderWithBlend(spriteBatch, bounds, BlendState.NonPremultiplied, 2);
    }

    private static void RenderWithBlend(SpriteBatch sb, CameraBounds bounds,
BlendState blend, byte blendMode)
    {
        sb.Begin(SpriteSortMode.Deferred, blend);
        _buffer.ForEachActive((index, ref p) =>
        {
            if (p.BlendMode != blendMode) return;

```

```

        // Culling: no dibujar si esta fuera de pantalla
        if (!bounds.IsVisible(p.Position)) return;

        Texture2D tex = _textures[p.TextureId];
        if (tex == null) return;

        Color color = UnpackColor(p.PackedColor);
        Vector2 screenPos = p.Position - Main.screenPosition;

        sb.Draw(tex, screenPos, null, color, p.Rotation,
                tex.Size() / 2f, p.Scale, SpriteEffects.None, 0f);
    });
    sb.End();
}

private static Color UnpackColor(uint packed) =>
    new Color(
        (byte)(packed & 0xFF),
        (byte)((packed >> 8) & 0xFF),
        (byte)((packed >> 16) & 0xFF),
        (byte)((packed >> 24) & 0xFF)
    );
}

```

15. Render Pipeline y Sistema de Layers

15.1 Layers como prioridades numericas

En lugar de un enum con valores hardcoded, esta libreria usa prioridades numericas (ushort) para indicar el orden de render. Valores mas bajos se renderizan antes (mas al fondo). El usuario puede especificar cualquier valor entre 0 y 65535. Se definen constantes pre-establecidas que mapean a los hooks de tModLoader para conveniencia.

```

public static class LayerPriorities // Renombrado para evitar conflicto con el
    campo del struct
{
    public const ushort Background      = 100; // Detras de todo
    public const ushort BeforeWalls     = 200; // Background walls
    public const ushort BeforeTiles     = 300; // Tiles no solidos
    public const ushort BeforeNPCsBehind = 400; // NPCs detras de tiles
    public const ushort BeforeSolidTiles = 500; // Tiles solidos
    public const ushort BeforeNPCs      = 600; // NPCs
    public const ushort BeforeProjectiles = 700; // Projectiles
    public const ushort BeforePlayers   = 800; // Players
    public const ushort BeforeItems     = 900; // Items dropped
    public const ushort BeforeRain      = 1000; // Lluvia
    public const ushort BeforeGore      = 1100; // Gore
    public const ushort BeforeDust      = 1200; // Dusts vanilla
    public const ushort AfterDust       = 1300; // Encima de dusts
    public const ushort BeforeInterface = 1400; // Antes de UI
    public const ushort AfterInterface  = 1500; // Encima de UI

    // El usuario puede usar cualquier valor, ej: 750 para entre Projectiles y
    Players
}

```

15.2 Hook en el pipeline de tModLoader

tModLoader expone hooks en ModSystem que se ejecutan en puntos específicos del pipeline de render. Para inyectar partículas en un punto, se overridea el hook correspondiente y se filtra por LayerPriority. Esto permite que las partículas se renderizen en el orden correcto relativo a NPCs, proyectiles, players, etc.

```
public class ParticleRenderSystem : ModSystem
{
    public override void PreUpdateDusts()
    {
        // Update de partículas: despues de que NPCs y proyectiles se actualizaron
        // pero antes de que se renderizen
        ParticleManager.Update();
    }

    public override void PostDrawTiles()
    {
        // Render partículas con layer <= BeforeSolidTiles
        RenderLayerRange(Main.spriteBatch, 0, LayerPriority.BeforeSolidTiles);
    }

    public override void PostDrawNPCs()
    {
        RenderLayerRange(Main.spriteBatch, LayerPriority.BeforeNPCs,
LayerPriorities.BeforePlayers);
    }

    public override void PostDrawProjectiles()
    {
        RenderLayerRange(Main.spriteBatch, LayerPriorities.BeforeProjectiles,
LayerPriorities.BeforePlayers);
    }

    public override void PostDrawPlayers()
    {
        RenderLayerRange(Main.spriteBatch, LayerPriorities.BeforePlayers,
LayerPriority.BeforeDust);
    }

    public override void PostDrawInterface()
    {
        RenderLayerRange(Main.spriteBatch, LayerPriority.BeforeInterface,
ushort.MaxValue);
    }

    private void RenderLayerRange(SpriteBatch sb, ushort minLayer, ushort
maxLayer)
    {
        // Implementacion simplificada: filtrar partículas por layer
        // En la implementacion real, se agruparian por blend mode tambien
        var bounds = new CameraBounds(Main.screenPosition, new
Vector2(Main.screenWidth, Main.screenHeight));

        sb.Begin(SpriteSortMode.Deferred, BlendState.AlphaBlend);
        ParticleManager.RenderInRange(sb, bounds, minLayer, maxLayer, 0);
        sb.End();

        sb.Begin(SpriteSortMode.Deferred, BlendState.Additive);
        ParticleManager.RenderInRange(sb, bounds, minLayer, maxLayer, 1);
        sb.End();
    }
}
```

```
}
```

16. Culling y Optimizaciones de Performance

16.1 Frustum culling

Frustum culling significa no dibujar partículas que están fuera de la pantalla. Para cada partícula, se verifica si su posición está dentro del rectángulo visible (screenPosition a screenPosition + screenSize). Si no está, se salta el draw call. Esto es especialmente importante cuando hay miles de partículas activas pero solo unas cientos son visibles.

```
public readonly struct CameraBounds
{
    public Vector2 TopLeft { get; }
    public Vector2 Size { get; }
    public Vector2 BottomRight => TopLeft + Size;

    public CameraBounds(Vector2 topLeft, Vector2 size)
    {
        TopLeft = topLeft;
        Size = size;
    }

    /// <summary>
    /// Verifica si una posición en mundo está dentro del área visible de la
    cámara.
    /// Incluye un margen para partículas grandes que pueden estar parcialmente
    fuera.
    /// </summary>
    public bool IsVisible(Vector2 worldPos, float margin = 64f)
    {
        return worldPos.X >= TopLeft.X - margin &&
            worldPos.X <= BottomRight.X + margin &&
            worldPos.Y >= TopLeft.Y - margin &&
            worldPos.Y <= BottomRight.Y + margin;
    }
}
```

16.2 Batching por blend mode

El BlendState se establece al llamar Begin() y se mantiene durante todo el batch; cambiarlo requiere un nuevo par Begin/End que fuerza flush. Para minimizar este overhead, se agrupan partículas por blend mode y se hace una pasada por cada uno. Si hay 3 blend modes (Alpha, Additive, NonPremultiplied) y 1000 partículas, en lugar de 1000 state changes se hacen solo 3.

16.3 SpriteSortMode.Deferred vs Immediate

SpriteSortMode.Deferred agrupa draw calls internamente y los envia al GPU en batch al llamar End(). Es el modo default y mas eficiente para la mayoria de casos. SpriteSortMode.Immediate envia cada draw call inmediatamente al GPU, util cuando se cambia textura o shader por draw call, pero mucho mas lento. Para partículas, Deferred es lo correcto.

16.4 Texture atlas consideration

Un texture atlas combina multiples texturas pequeñas en una sola grande. Esto reduce el overhead de cambiar texturas entre draw calls. Para una libreria de particulas, se podria ofrecer un atlas builder que combine todas las texturas registradas en una sola, y cada partícula especifica el rectangulo fuente dentro del atlas. Esto es una optimizacion avanzada que no se implementa en la version inicial pero se deja como roadmap.

17. Multiplayer y Sincronizacion

17.1 El problema de la sincronizacion

En multiplayer, cada cliente corre su propia instancia del juego. Si un cliente spawnnea una partícula, los otros clientes no la veran a menos que se sincronice via paquetes de red. Sin embargo, las partículas son puramente visuales: no afectan gameplay. Por tanto, hay dos enfoques posibles: sincronizar spawns (mas ancho de banda, todos ven lo mismo) o no sincronizar (cada cliente ve sus propias partículas). El segundo enfoque es mas eficiente y es el adoptado por la mayoria de mods.

17.2 Enfoque adoptado: client-side only

Esta libreria adopta el enfoque client-side only: las partículas se spawnnean y renderizan solo en el cliente que las genera. Si el servidor spawnnea una partícula (porque un NPC recibio dano, por ejemplo), el servidor NO la envia a los clientes; en su lugar, cada cliente detecta el evento (via OnHitNPC hook) y spawnnea su propia partícula localmente. Esto es eficiente y suficiente para VFX.

17.3 Hooks multiplayer-safe

Algunos hooks de tModLoader se ejecutan en servidor y otros en cliente. Para evitar bugs, la libreria debe chequear Main.netMode antes de spawnnear partículas. Main.netMode puede ser 0 (singleplayer), 1 (multiplayer client), o 2 (server). Las partículas solo se spawnnean en netMode 0 o 1.

```
public static bool CanSpawnParticles => Main.netMode != 2; // No en server

// En un GlobalProjectile.OnHitNPC:
public override void OnHitNPC(Projectile projectile, NPC target, NPC.HitInfo hit,
int damageDone)
{
    if (!CanSpawnParticles) return; // Skip en server

    // Spawnear partículas de impacto
    ParticleManager.SpawnShape(target.Center, ShapeDescriptor.Circle(20f), 10,
```

```
new ParticleData { /* ... */ };
```

17.4 Sincronizacion opcional para eventos criticos

Para eventos criticos que TODOS los clientes deben ver (como un boss spawn), se puede sincronizar via ModPacket. El servidor envia un paquete con la posicion y el tipo de efecto, y cada cliente lo recibe y spawna las partículas localmente. Esto es raramente necesario para VFX, pero conviene conocer el patron.

18. Hooks de tModLoader para VFX

18.1 ModSystem hooks

ModSystem es la clase base para sistemas globales del mod. Expone hooks que se ejecutan en puntos especificos del ciclo de vida del juego. Para una libreria de particulas, los hooks mas relevantes son: OnModLoad (inicializacion), PostUpdateWorld (update de partículas), PostDrawTiles/PostDrawNPCs/PostDrawProjectiles/PostDrawPlayers/PostDrawInterface (render en distintos puntos del pipeline), y PostSetupContent (cross-mod setup).

Hook	Cuando se ejecuta	Uso tipico
OnModLoad	Una vez al cargar el mod	Inicializar ParticleManager
Unload	Al descargar el mod	Liberar recursos
PostSetupContent	Despues de cargar todos los mods	Cross-mod setup
PreUpdateDusts	Antes de update de Dusts vanilla	Update de partículas
PostUpdateWorld	Despues de update del mundo	Update alternativo
PostDrawTiles	Despues de renderizar tiles	Render partículas early layers
PostDrawNPCs	Despues de renderizar NPCs	Render layer BeforeNPCs-Players
PostDrawProjectiles	Despues de projectiles	Render layer projectiles
PostDrawPlayers	Despues de players	Render layer players
PostDrawInterface	Despues de UI	Render layer interface
ModifyScreenPosition	Antes de calcular screen pos	Screen shake integration

18.2 GlobalItem hooks para VFX de items

GlobalItem permite modificar comportamiento de TODOS los items (vanilla y de otros mods). Para VFX, los hooks utiles son: HoldItem (efectos mientras se sostiene), UseItem (efectos al usar), OnHitNPC (partículas al golpear), y Shoot (partículas al disparar).

18.3 GlobalProjectile hooks

GlobalProjectile permite modificar todos los proyectiles. Hooks utiles: AI (partículas mientras el proyectil esta activo), OnHitNPC (impacto), Kill (muerte), PreDraw (trail visual). El patron clasico es spawnear 1-2 partículas por frame en AI() para crear un trail.

18.4 ModPlayer hooks

ModPlayer permite modificar jugadores. Hooks utiles: PreUpdate (efectos pasivos cada frame), OnHitNPCWithItem/OnHitNPCWithProj (partículas al jugador golpear), PostUpdate (estados visuales), y el sistema PlayerDrawLayer para custom drawing sobre el jugador (reemplaza al antiguo ModifyDrawLayers eliminado en 1.4.4).

19. Implementacion de ModSystem, GlobalItem, GlobalProjectile

19.1 ParticleSystem (ModSystem principal)

```
using Terraria.ModLoader;
using Terraria;
using Microsoft.Xna.Framework;
using Microsoft.Xna.Framework.Graphics;

namespace MiParticleLib.Common
{
    public class ParticleSystem : ModSystem
    {
        public override void OnModLoad()
        {
            if (Main.dedicatedServer) return; // No render en server

            ParticleManager.Initialize(MiParticleLib.Config.MaxParticles);

            // Registrar texturas built-in
            ParticleManager.RegisterTexture("MiParticleLib/Content/Textures/Default");
            ParticleManager.RegisterTexture("MiParticleLib/Content/Textures/SoftGlow");
            ParticleManager.RegisterTexture("MiParticleLib/Content/Textures/Spark");
            ParticleManager.RegisterTexture("MiParticleLib/Content/Textures/Smoke");
            ParticleManager.RegisterTexture("MiParticleLib/Content/Textures/Trail");

            // Cargar shaders
            ShaderLoader.LoadAll(Mod);
        }

        public override void Unload()
        {
            ParticleManager.Shutdown();
        }

        public override void PreUpdateDusts()
        {
            if (Main.dedicatedServer) return;
            ParticleManager.Update();
        }

        public override void PostDrawTiles()
        {
            if (Main.dedicatedServer) return;
            var bounds = new CameraBounds(Main.screenPosition, new Vector2(Main.screenWidth, Main.screenHeight));
            ParticleManager.Render(Main.spriteBatch, bounds);
        }

        // Hook para integracion con screen shake
        public override void ModifyScreenPosition()
        {
            // Aqui se podria aplicar offset por screen shake activo
        }
    }
}
```



```

    }
}

```

19.2 GlobalProjectile con trail automatico

```

public class GlobalParticleProjectile : GlobalProjectile
{
    public override bool InstancePerEntity => true;

    public bool HasCustomTrail { get; set; }
    public ushort TrailTextureId { get; set; }
    public Color TrailColor { get; set; } = Color.White;
    public float TrailIntensity { get; set; } = 1f;

    public override void AI(Projectile projectile)
    {
        if (!HasCustomTrail || Main.dedicatedServer) return;

        // Spawnear 1 partícula trail por frame
        var data = new ParticleData
        {
            Position = projectile.Center,
            Velocity = -projectile.velocity * 0.1f,
            Scale = new Vector2(TrailIntensity),
            PackedColor = PackColor(TrailColor),
            TimeLeft = 20,
            Duration = 20,
            TextureId = TrailTextureId,
            BlendMode = 1, // Additive
            LayerPriority = LayerPriorities.BeforeProjectiles,
        };
        data.EnableComponent(ComponentFlag.FadeOut);
        data.EnableComponent(ComponentFlag.ScaleDown);

        ParticleManager.Spawn(data);
    }

    public override void OnHitNPC(Projectile projectile, NPC target, NPC.HitInfo hit, int damageDone)
    {
        if (Main.dedicatedServer) return;

        // Impact burst
        ParticleManager.SpawnShape(target.Center, ShapeDescriptor.Circle(15f), 8,
            new ParticleData
            {
                Velocity = Vector2.Zero,
                Scale = Vector2.One,
                PackedColor = PackColor(Color.Yellow),
                TimeLeft = 30,
                Duration = 30,
                TextureId = 2, // Spark
                BlendMode = 1, // Additive
            });
    }

    private static uint PackColor(Color c) =>
        (uint)c.R | ((uint)c.G << 8) | ((uint)c.B << 16) | ((uint)c.A << 24);
}

```

20. PlayerLayer y Custom Drawing

20.1 Que es PlayerLayer

PlayerLayer es el sistema de tModLoader para dibujar cosas sobre el jugador. Permite anadir sprites custom que siguen al jugador (alas, auras, accesorios visuales, efectos de buff). En tModLoader 1.4.4, se usa PlayerDrawLayer (clase nueva) en lugar del antiguo PlayerLayer.

20.2 PlayerDrawLayer para auras

```
public class AuraDrawLayer : PlayerDrawLayer
{
    public override Position GetDefaultPosition() => new
    AfterParent(PlayerDrawLayers.ArmOverItem);

    protected override void Draw(ref PlayerDrawSet drawInfo)
    {
        Player player = drawInfo.drawPlayer;
        var modPlayer = player.GetModPlayer<AuraPlayer>();

        if (!modPlayer.HasAura) return;

        Texture2D aura =
        ModContent.Request<Texture2D>("MiParticleLib/Content/Textures/Aura").Value;
        Vector2 pos = player.Center - Main.screenPosition;

        // Aura que pulsa con el tiempo
        float pulse = 1f + (float)Math.Sin(Main.GlobalTimeWrappedHourly * 3f) *
0.1f;
        Color color = modPlayer.AuraColor * 0.5f;

        drawInfo.DrawDataCache.Add(new DrawData(
            aura, pos, null, color, 0f,
            aura.Size() / 2f, pulse, SpriteEffects.None, 0
        ));
    }
}
```

20.3 ModPlayer con estado de aura

```
public class AuraPlayer : ModPlayer
{
    public bool HasAura { get; set; }
    public Color AuraColor { get; set; } = Color.Cyan;

    public override void ResetEffects()
    {
        HasAura = false; // Reset cada frame
    }

    public override void PostUpdate()
    {
        if (HasAura && !Main.dedicatedServer)
        {
            // Spawnear partículas orbitales
            if (Main.rand.NextBool(5))
            {
                float angle = Main.rand.NextFloat(0f, MathHelper.TwoPi);
                Vector2 offset = new Vector2(40f, 0f).RotatedBy(angle);

                var data = new ParticleData
                {
                    Position = Player.Center + offset,
                    Velocity = Vector2.Zero,
                    Scale = Vector2.One * 0.5f,
                    PackedColor = PackColor(AuraColor),
                    TimeLeft = 30,
                    Duration = 30,
                    TextureId = 1, // SoftGlow
                    BlendMode = 1, // Additive
                };
                data.EnableComponent(ComponentFlag.FadeOut);
                ParticleManager.Spawn(data);
            }
        }
    }

    private static uint PackColor(Color c) =>
        (uint)c.R | ((uint)c.G << 8) | ((uint)c.B << 16) | ((uint)c.A << 24);
}
```

21. Asset Pipeline: Carga de Texturas

21.1 Como tModLoader carga texturas

tModLoader usa `ModContent.Request<T>(path)` para cargar assets. El path es relativo al mod sin extension: "MiMod/Content/Textures/MiTextura" carga `MiMod/Content/Textures/MiTextura.png`. La carga es lazy por defecto (se carga al primer uso) y cached. Para forzar carga inmediata, se usa `.Wait()` o se carga en `OnModLoad`.

21.2 Registro centralizado de texturas

```
public static class TextureRegistry
{
    private static readonly Dictionary<string, ushort> _pathToId = new();
    private static readonly List<Texture2D> _textures = new();
    private static readonly List<string> _paths = new();

    public static ushort Default { get; private set; }
    public static ushort SoftGlow { get; private set; }
    public static ushort Spark { get; private set; }
    public static ushort Smoke { get; private set; }
    public static ushort Trail { get; private set; }

    public static void Initialize()
    {
        Default = Register("MiParticleLib/Content/Textures/Default");
        SoftGlow = Register("MiParticleLib/Content/Textures/SoftGlow");
        Spark = Register("MiParticleLib/Content/Textures/Spark");
        Smoke = Register("MiParticleLib/Content/Textures/Smoke");
        Trail = Register("MiParticleLib/Content/Textures/Trail");
    }

    public static ushort Register(string path)
    {
        if (_pathToId.TryGetValue(path, out ushort id))
            return id;

        id = (ushort)_textures.Count;
        _textures.Add(ModContent.Request<Texture2D>(path).Value);
        _paths.Add(path);
        _pathToId[path] = id;
        return id;
    }

    public static Texture2D Get(ushort id) =>
        id < _textures.Count ? _textures[id] : _textures[Default];

    public static void Unload()
    {
        _pathToId.Clear();
        _textures.Clear();
        _paths.Clear();
    }
}
```

22. Asset Pipeline: Carga de Shaders

22.1 Compilacion de shaders .fx a .xnb

Los shaders HLSL se escriben en archivos .fx y se compilan a .xnb usando MonoGame Content Pipeline (mgcb) o la herramienta EasyXnb.exe. El .xnb es un formato binario que tModLoader carga como Effect object. El proceso de compilacion valida sintaxis HLSL, optimiza el shader, y lo empaqueta para la plataforma target.

22.2 Carga de shaders en tModLoader

```
public static class ShaderLoader
{
    public static Effect ParticleShader { get; private set; }
    public static Effect InstancedShader { get; private set; }
    public static Effect ShapeShader { get; private set; }

    public static void LoadAll(Mod mod)
    {
        ParticleShader = LoadEffect(mod, "Content/Effects/ParticleShader");
        InstancedShader = LoadEffect(mod, "Content/Effects/InstancedShader");
        ShapeShader = LoadEffect(mod, "Content/Effects/ShapeShader");
    }

    private static Effect LoadEffect(Mod mod, string path)
    {
        try
        {
            return mod.Assets.Request<Effect>(path).Value;
        }
        catch (Exception e)
        {
            Mod.Logger.Error($"Failed to load shader {path}: {e}");
            return null;
        }
    }
}
```

22.3 Aplicar un shader en SpriteBatch

```
// Usar SpriteSortMode.Immediate para que el shader se aplique inmediatamente
spriteBatch.Begin(SpriteSortMode.Immediate, BlendState.AlphaBlend);

// Aplicar el shader
if (ShaderLoader.ParticleShader != null)
{
    ShaderLoader.ParticleShader.Parameters["Time"].SetValue(Main.GlobalTimeWrapped
Hourly);
    ShaderLoader.ParticleShader.Parameters["Tint"].SetValue(Color.Red.ToVector4())
;
    ShaderLoader.ParticleShader.CurrentTechnique.Passes[0].Apply();
}

// Dibujar partículas (el shader se aplica a todas)
foreach (var particle in visibleParticles)
{
    spriteBatch.Draw(texture, particle.Position - Main.screenPosition, color);
}

spriteBatch.End();
```

23. Shaders HLSL: Introduccion

23.1 Que es HLSL

HLSL (High-Level Shading Language) es el lenguaje de shaders de Microsoft usado por DirectX. MonoGame, al estar basado en XNA, usa HLSL para sus shaders. Un shader es un pequeno programa que se ejecuta en el GPU y procesa vertices (vertex shader) o pixeles (pixel shader). Para una libreria de particulas, los shaders se usan para efectos visuales avanzados que no se pueden lograr con SpriteBatch basico: distorsion, color grading procedural, glow amplificado, instancing masivo.

23.2 Estructura de un shader .fx

Un archivo .fx tiene tres secciones principales: variables globales (parametros que el codigo C# setea antes de aplicar el shader), samplers (texturas), y techniques/passes (los puntos de entrada del shader). Un technique agrupa uno o mas passes, y cada pass define un vertex shader y un pixel shader.

```
// Variables globales - seteadas desde C# via
shader.Parameters["Nombre"].SetValue(valor)
float Time;
float4 Tint;
matrix TransformMatrix;

// Sampler para textura
texture Texture;
sampler TextureSampler = sampler_state
{
    Texture = (Texture);
    magfilter = LINEAR;
    minfilter = LINEAR;
    mipfilter = LINEAR;
    AddressU = Clamp;
    AddressV = Clamp;
};

// Structs de entrada/salida del vertex shader
struct VertexShaderInput
{
    float4 Position : POSITION0;
    float2 TexCoords : TEXCOORD0;
    float4 Color : COLOR0;
};

struct VertexShaderOutput
{
    float4 Position : POSITION0;
    float2 TexCoords : TEXCOORD0;
    float4 Color : COLOR0;
};

// Vertex shader: transforma posicion y pasa-through UVs y color
VertexShaderOutput VertexShaderFunction(VertexShaderInput input)
{
    VertexShaderOutput output;
    output.Position = mul(input.Position, TransformMatrix);
    output.TexCoords = input.TexCoords;
    output.Color = input.Color;
    return output;
}
```

```
// Pixel shader: samplea textura y aplica tint
float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    float4 texColor = tex2D(TextureSampler, input.TexCoords);
    return texColor * input.Color * Tint;
}

// Technique: punto de entrada del shader
technique DefaultTechnique
{
    pass Pass0
    {
        VertexShader = compile vs_2_0 VertexShaderFunction();
        PixelShader = compile ps_2_0 PixelShaderFunction();
    }
};
```

23.3 Compilacion a .xnb

Para que tModLoader cargue un shader, el archivo .fx debe compilarse a .xnb usando MonoGame Content Pipeline. Hay dos formas: usar mgcb (command line) o EasyXnb.exe (GUI). El comando basico es: mgcb /build:MiShader.fx /outputDir:. /platform:Windows. Esto genera MiShader.xnb que se coloca en Content/Effects/ del mod.

24. Shader de Quad Particle

24.1 Vertex format custom

Para renderizar partículas con todos sus atributos (posicion, velocidad, color, scale, rotacion, tiempo), se necesita un vertex format custom. MonoGame permite definir structs con semantics HLSL especificas. El shader recibe estos datos por vertice y los procesa.

24.2 Código HLSL completo

```
// ParticleShader.fx - Shader para renderizar quad particles con todos los
// atributos

#define PI 3.14159265
#define TAU 6.28318531

matrix TransformMatrix;
float Time;
float2 ScreenPosition;
float2 Offset;

bool Fade;
bool UseCurve;

texture Texture;
sampler TextureSampler = sampler_state
{
    Texture = (Texture);
    magfilter = LINEAR; minfilter = LINEAR; mipfilter = LINEAR;
    AddressU = Clamp; AddressV = Clamp;
};

texture Curve;
sampler CurveSampler = sampler_state
{
    Texture = (Curve);
    magfilter = LINEAR; minfilter = LINEAR; mipfilter = LINEAR;
    AddressU = Clamp; AddressV = Clamp;
};

// Vertex input: todos los datos de una partícula quad
struct GVertexShaderInput
{
    float4 Position : POSITION0;
    float4 TexCoords : TEXCOORD0;
    float4 StartColor : COLOR0;
    float4 EndColor : COLOR1;
    float4 Velocity : NORMAL0;          // XY velocity | ZW acceleration
    float2 Acceleration : NORMAL1;
    float2 Size : NORMAL2;
    float4 Scale : NORMAL3;             // XY scale | ZW velocity
    float4 Rotation : NORMAL4;          // XY corner | Z rotation | W velocity
    float4 DepthTime : NORMAL5;         // X depth | Y velocity | Z time
};

struct GVertexShaderOutput
{
    float4 Position : POSITION0;
    float2 TexCoords : TEXCOORD0;
    float4 Color : COLOR0;
};

GVertexShaderOutput VertexShaderFunction(GVertexShaderInput input)
{
    GVertexShaderOutput output;

    // Calcular tiempo normalizado 0-1
    float t = input.DepthTime.z / 60.0; // Asumiendo duration de 60 ticks

    // Interpolar color
    float4 color = lerp(input.StartColor, input.EndColor, t);
```



```

// Calcular posicion final con velocidad y aceleracion
float2 pos = input.Position.xy;
pos += input.Velocity.xy * input.DepthTime.z;
pos += 0.5 * input.Acceleration * input.DepthTime.z * input.DepthTime.z;

// Aplicar scale (con animacion)
float2 scale = input.Scale.xy;
scale += input.Scale.zw * input.DepthTime.z; // Scale velocity

// Aplicar rotacion (con velocidad angular)
float rotation = input.Rotation.z + input.Rotation.w * input.DepthTime.z;

// Rotar la posicion del corner alrededor del centro
float2 corner = input.Rotation.xy;
float cosR = cos(rotation);
float sinR = sin(rotation);
float2 rotatedCorner = float2(
    corner.x * cosR - corner.y * sinR,
    corner.x * sinR + corner.y * cosR
);

// Posicion final en screen space
float2 finalPos = pos + rotatedCorner * scale * Size;
finalPos -= ScreenPosition + Offset;

output.Position = mul(float4(finalPos, input.DepthTime.x, 1.0),
TransformMatrix);
output.TexCoords = input.TexCoords.xy;
output.Color = color;

return output;
}

float4 PixelShaderFunction(GVertexShaderOutput input) : COLOR0
{
    float4 texColor = tex2D(TextureSampler, input.TexCoords);
    float4 finalColor = texColor * input.Color;

    // Aplicar fade opcional
    if (Fade)
    {
        float fade = tex2D(CurveSampler, float2(input.Color.a, 0.5)).r;
        finalColor.a *= fade;
    }

    return finalColor;
}

technique DefaultTechnique
{
    pass Pass0
    {
        VertexShader = compile vs_2_0 VertexShaderFunction();
        PixelShader = compile ps_2_0 PixelShaderFunction();
    }
};

```

25. Shader de Instanced Particles (GPU)

25.1 Que es instancing

Instancing es una tecnica de rendering que permite dibujar miles de instancias de la misma geometria en un solo draw call. Para cada instancia, se pasan datos custom (posicion, color, scale) via vertex streams separados. Esto reduce drasticamente el overhead de CPU->GPU communication. Para una libreria de particulas, instancing permite renderizar 10000+ particulas en un solo draw call en lugar de 10000 draw calls separados.

25.2 Código HLSL de instanced shader

```
// InstancedShader.fx - Renderiza miles de partículas en un draw call

sampler Texture : register(s0);
matrix Transform;
float2 Offset;

// Vertex base (un quad: 4 vertices)
struct VertexShaderInput
{
    float2 Position : POSITION0;
    float2 Texture : TEXCOORD0;
};

// Datos por instancia (una partícula)
struct Particle
{
    float4 Position_Scale : NORMAL0;    // XY position, ZW scale
    float2 Rotation_Depth : NORMAL1;    // X rotation, Y depth
    float4 Color : COLOR0;
};

// Helper: matriz de traslacion
float4x4 TranslationMatrix(float2 position)
{
    return float4x4(
        1, 0, 0, 0,
        0, 1, 0, 0,
        0, 0, 1, 0,
        position.x, position.y, 0, 1
    );
}

// Helper: matriz de rotacion
float4x4 RotationMatrix(float rotation)
{
    float cosRoll = cos(rotation);
    float sinRoll = sin(rotation);
    return float4x4(
        cosRoll, sinRoll, 0, 0,
        -sinRoll, cosRoll, 0, 0,
        0, 0, 1, 0,
        0, 0, 0, 1
    );
}

// Helper: matriz de escala
float4x4 ScaleMatrix(float2 scale)
{
    return float4x4(
        scale.x, 0, 0, 0,
        0, scale.y, 0, 0,
        0, 0, 1, 0,
        0, 0, 0, 1
    );
}

struct VertexShaderOutput
{
    float4 Position : POSITION0;
    float4 Color : COLOR0;
    float2 Texture : TEXCOORD0;
};
```

```

// Vertex shader: recibe vertice base + instancia
VertexShaderOutput Vertex(VertexShaderInput input, Particle instance)
{
    VertexShaderOutput output;

    // Construir matriz de transformacion
    float4x4 translation = TranslationMatrix(instance.Position_Scale.xy - Offset);
    float4x4 rotation = RotationMatrix(instance.Rotation_Depth.x);
    float4x4 scale = ScaleMatrix(instance.Position_Scale.zw);

    // Combinar: T * R * S * vertex
    float4x4 world = mul(mul(translation, rotation), scale);
    float4 worldPos = mul(float4(input.Position, 0, 1), world);

    output.Position = mul(worldPos, Transform);
    output.Color = instance.Color;
    output.Texture = input.Texture;

    return output;
}

float4 Pixel(VertexShaderOutput input) : COLOR0
{
    float4 texColor = tex2D(Texture, input.Texture);
    return texColor * input.Color;
}

technique DefaultTechnique
{
    pass Pass0
    {
        VertexShader = compile vs_3_0 Vertex();
        PixelShader = compile ps_3_0 Pixel();
    }
};

```

26. Shader de Formas Procedurales (SDF)

26.1 Que es SDF

SDF (Signed Distance Field) es una tecnica para renderizar formas procedurales sin textura. Para cada pixel, se calcula la distancia al borde de la forma. Si la distancia es negativa, el pixel esta dentro; si es positiva, fuera. Esto permite dibujar circulos, rectangulos redondeados, y formas complejas con calidad vectorial sin aliasing.

26.2 Shader Shape.fx

```
// ShapeShader.fx - Renderiza formas procedurales via SDF

#define PI 3.14159265

matrix UIScaleMatrix;
float4 OutlineColor = float4(1, 0, 0, 1);
float OutlineThickness;
float Radius;
float2 Size;

struct VertexShaderInput
{
    float4 Position : POSITION;
    float2 TexCoords : TEXCOORD0;
    float4 Color : COLOR0;
};

struct VertexShaderOutput
{
    float4 Position : POSITION;
    float2 TexCoords : TEXCOORD0;
    float4 Color : COLOR0;
};

// SDF para rectangulo redondeado
float RoundedRectangle(float2 center, float2 size, float radius)
{
    return length(max(abs(center) - (size / 2) + radius, 0)) - radius;
}

// SDF para circulo
float Circle(float2 center, float radius)
{
    return length(center) - radius;
}

VertexShaderOutput VertexShaderFunction(VertexShaderInput input)
{
    VertexShaderOutput output;
    output.Position = mul(input.Position, UIScaleMatrix);
    output.TexCoords = input.TexCoords;
    output.Color = input.Color;
    return output;
}

float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    // Convertir UV a pixel coordinates
    float2 position = float2(input.TexCoords.x * Size.x, input.TexCoords.y *
Size.y);

    // Calcular distancia al borde
    float distance = RoundedRectangle(position - (Size / 2), Size, Radius);

    // Clip pixeles fuera de la forma
    clip(-distance);

    // Aplicar outline si esta cerca del borde
    if (Radius > 0)
    {
        return distance <= -OutlineThickness ? input.Color : OutlineColor;
    }
}
```

```

    else
    {
        // Modo box (sin redondear esquinas)
        return (position.x <= OutlineThickness ||
                position.x >= Size.x - OutlineThickness ||
                position.y <= OutlineThickness ||
                position.y >= Size.y - OutlineThickness)
            ? OutlineColor : input.Color;
    }
}

technique DefaultTechnique
{
    pass CirclePass
    {
        PixelShader = compile ps_3_0 PixelShaderFunction();
    }
};

```

27. Shader de Bloom

27.1 Que es bloom

Bloom es un efecto post-processing que hace que las areas brillantes de la imagen "sangren" luz hacia las areas oscuras adyacentes. Es lo que hace que las luces se vean "reales" en juegos modernos. Implementarlo requiere: extraer las areas brillantes, blurlearlas, y combinarlas con la imagen original.

27.2 Bloom shader HLSL

```
// BloomShader.fx

sampler ScreenTexture : register(s0);
float BloomThreshold = 0.8;
float BloomIntensity = 1.0;
float2 TexelSize;

// Paso 1: extraer areas brillantes
float4 ExtractBright(VertexShaderOutput input) : COLOR0
{
    float4 color = tex2D(ScreenTexture, input.TexCoords);
    float brightness = dot(color.rgb, float3(0.299, 0.587, 0.114)); // Luma
    return brightness > BloomThreshold ? color * BloomIntensity : float4(0, 0, 0, 0);
}

// Paso 2: blur horizontal
float4 BlurHorizontal(VertexShaderOutput input) : COLOR0
{
    float4 color = float4(0, 0, 0, 0);
    float weights[5] = {0.227027, 0.1945946, 0.1216216, 0.054054, 0.016216};

    color += tex2D(ScreenTexture, input.TexCoords) * weights[0];
    for (int i = 1; i < 5; i++)
    {
        color += tex2D(ScreenTexture, input.TexCoords + float2(TexelSize.x * i, 0)) * weights[i];
        color += tex2D(ScreenTexture, input.TexCoords - float2(TexelSize.x * i, 0)) * weights[i];
    }
    return color;
}

// Paso 3: blur vertical + combinar con original
float4 CombineBloom(VertexShaderOutput input) : COLOR0
{
    float4 original = tex2D(ScreenTexture, input.TexCoords);
    float4 blurred = float4(0, 0, 0, 0);

    float weights[5] = {0.227027, 0.1945946, 0.1216216, 0.054054, 0.016216};
    blurred += tex2D(ScreenTexture, input.TexCoords) * weights[0];
    for (int i = 1; i < 5; i++)
    {
        blurred += tex2D(ScreenTexture, input.TexCoords + float2(0, TexelSize.y * i)) * weights[i];
        blurred += tex2D(ScreenTexture, input.TexCoords - float2(0, TexelSize.y * i)) * weights[i];
    }

    return original + blurred * BloomIntensity;
}

technique Bloom
{
    pass Extract { PixelShader = compile ps_3_0 ExtractBright(); }
    pass BlurH { PixelShader = compile ps_3_0 BlurHorizontal(); }
    pass Combine { PixelShader = compile ps_3_0 CombineBloom(); }
};
```

28. Shader de Chromatic Aberration

28.1 Que es chromatic aberration

Chromatic aberration simula el defecto optico de las lentes donde los canales de color (R, G, B) no convergen en el mismo punto. Visualmente crea un halo RGB en los bordes de objetos brillantes. Es muy efectivo para momentos dramaticos (ultimates, boss spawns, explosiones).

28.2 Shader HLSL

```
// ChromaticAberration.fx

sampler ScreenTexture : register(s0);
float AberrationAmount = 0.005;
float2 Center = float2(0.5, 0.5);

float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    // Vector del centro a este pixel
    float2 dir = input.TexCoords - Center;
    float dist = length(dir);
    dir = normalize(dir);

    // Cantidad de aberration basada en distancia al centro
    float amount = AberrationAmount * dist;

    // Samplear cada canal con offset diferente
    float r = tex2D(ScreenTexture, input.TexCoords + dir * amount).r;
    float g = tex2D(ScreenTexture, input.TexCoords).g;
    float b = tex2D(ScreenTexture, input.TexCoords - dir * amount).b;
    float a = tex2D(ScreenTexture, input.TexCoords).a;

    return float4(r, g, b, a);
}

technique DefaultTechnique
{
    pass Pass0
    {
        PixelShader = compile ps_3_0 PixelShaderFunction();
    }
};
```

29. Shader de Vignette

29.1 Vignette effect

Vignette oscurece los bordes de la imagen para crear un efecto de enfoque al centro. Es util para indicar baja vida, zonas peligrosas, o simplemente para dar un look cinematografico.

```
// VignetteShader.fx

sampler ScreenTexture : register(s0);
float VignetteIntensity = 0.5;
float VignetteRadius = 0.8;
```



```

float VignetteSoftness = 0.5;
float4 VignetteColor = float4(0, 0, 0, 1);

float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    float4 color = tex2D(ScreenTexture, input.TexCoords);

    // Distancia al centro (0-1)
    float2 center = float2(0.5, 0.5);
    float dist = distance(input.TexCoords, center);

    // Calcular vignette
    float vignette = smoothstep(VignetteRadius, VignetteRadius - VignetteSoftness,
dist);
    vignette = lerp(1.0, vignette, VignetteIntensity);

    // Aplicar
    return color * vignette + VignetteColor * (1.0 - vignette) *
VignetteIntensity;
}

technique DefaultTechnique
{
    pass Pass0 { PixelShader = compile ps_3_0 PixelShaderFunction(); }
};

```

30. Shader de Shockwave

30.1 Shockwave effect

Shockwave crea una onda expansiva visual desde un punto. Distorsiona la imagen en un anillo que se expande. Es efectivo para explosiones, slams de boss, y eventos dramaticos.

```

// ShockwaveShader.fx

sampler ScreenTexture : register(s0);
float2 ShockwaveCenter = float2(0.5, 0.5);
float ShockwaveRadius = 0.3;
float ShockwaveWidth = 0.05;
float ShockwaveStrength = 0.02;

float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    float2 uv = input.TexCoords;

    // Distancia al centro del shockwave
    float dist = distance(uv, ShockwaveCenter);

    // Calcular offset si estamos en el anillo del shockwave
    float diff = abs(dist - ShockwaveRadius);
    if (diff < ShockwaveWidth)
    {
        // Direccion radial
        float2 dir = normalize(uv - ShockwaveCenter);

        // Intensidad maxima en el centro del anillo, decae hacia los bordes
        float intensity = 1.0 - (diff / ShockwaveWidth);
        intensity = smoothstep(0.0, 1.0, intensity);
    }
}

```

```

        // Aplicar offset
        uv += dir * ShockwaveStrength * intensity;
    }

    return tex2D(ScreenTexture, uv);
}

technique DefaultTechnique
{
    pass Pass0 { PixelShader = compile ps_3_0 PixelShaderFunction(); }
};

```

31. Shader de Distortion / Heat Haze

31.1 Distorsion por calor

Simula la refraccion del aire caliente sobre fuego o lava. Usa una textura de ruido para desplazar los UVs de la imagen base, creando un efecto ondulante.

```

// DistortionShader.fx

sampler ScreenTexture : register(s0);
sampler NoiseTexture : register(s1);
float Time;
float DistortionStrength = 0.01;
float NoiseScale = 1.0;
float ScrollSpeed = 0.5;

float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    float2 uv = input.TexCoords;

    // Samplear textura de ruido con scroll vertical
    float2 noiseUV = uv * NoiseScale;
    noiseUV.y += Time * ScrollSpeed;
    float2 noise = tex2D(NoiseTexture, noiseUV).rg;

    // Centrar el ruido alrededor de 0
    noise = noise * 2.0 - 1.0;

    // Aplicar offset a UVs
    float2 distortedUV = uv + noise * DistortionStrength;

    return tex2D(ScreenTexture, distortedUV);
}

technique DefaultTechnique
{
    pass Pass0 { PixelShader = compile ps_3_0 PixelShaderFunction(); }
};

```

32. Shader de Color Grading

32.1 Color grading con LUT

Color grading aplica una paleta custom a toda la imagen. La forma mas eficiente es usar una LUT (Look-Up Table): una textura que mapea cada color de entrada a un color de salida. Esto permite efectos cinematicos como tono sepia, alto contraste, paleta de horror, etc.

```
// ColorGradingShader.fx

sampler ScreenTexture : register(s0);
sampler LUT : register(s1);
float LUTSize = 32.0; // 32x32x32 LUT

float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    float4 color = tex2D(ScreenTexture, input.TexCoords);

    // Normalizar color a rango 0-1
    color = saturate(color);

    // Calcular posicion en la LUT 3D
    // La LUT esta aplanada en una textura 2D: cada cuadro es un slice del azul
    float3 scaled = color.rgb * (LUTSize - 1.0);
    float b = scaled.z;

    // Posicion del slice en X
    float sliceWidth = 1.0 / LUTSize;
    float sliceX = (b + 0.5) * sliceWidth;

    // Posicion dentro del slice (R y G)
    float2 lutUV;
    lutUV.x = (scaled.x + 0.5) * sliceWidth + floor(b) * sliceWidth;
    lutUV.y = (scaled.y + 0.5) / LUTSize;

    // Samplear LUT
    return tex2D(LUT, lutUV);
}

technique DefaultTechnique
{
    pass Pass0 { PixelShader = compile ps_3_0 PixelShaderFunction(); }
};
```

33. Shader de Radial Blur

33.1 Radial blur effect

Radial blur desenfoca la imagen desde un punto central hacia afuera. Crea sensacion de velocidad o foco dramatico. Es efectivo para highlights de ultimate abilities.

```
// RadialBlurShader.fx

sampler ScreenTexture : register(s0);
float2 BlurCenter = float2(0.5, 0.5);
float BlurStrength = 0.1;
```

```

int BlurSamples = 8;

float4 PixelShaderFunction(VertexShaderOutput input) : COLOR0
{
    float2 uv = input.TexCoords;
    float2 dir = uv - BlurCenter;

    float4 color = float4(0, 0, 0, 0);
    float totalWeight = 0.0;

    for (int i = 0; i < BlurSamples; i++)
    {
        float t = i / (float)BlurSamples;
        float2 sampleUV = uv - dir * t * BlurStrength;
        float weight = 1.0 - t;
        color += tex2D(ScreenTexture, sampleUV) * weight;
        totalWeight += weight;
    }

    return color / totalWeight;
}

technique DefaultTechnique
{
    pass Pass0 { PixelShader = compile ps_3_0 PixelShaderFunction(); }
};

```

34. Compilacion de Shaders: EasyXnb y MGCB

34.1 EasyXnb.exe

EasyXnb.exe es una herramienta standalone incluida en ParticleLibrary que permite compilar shaders .fx a .xnb sin instalar MonoGame SDK completo. Para crear una herramienta equivalente para esta libreria, se puede empaquetar los DLLs necesarios (Microsoft.Xna.Framework.Content.Pipeline.dll, Microsoft.Xna.Framework.Graphics.dll) en una carpeta y usarlos via reflection o command line.

34.2 MGCB (MonoGame Content Builder)

MGCB es la herramienta oficial de MonoGame para compilar assets. Se instala con el MonoGame SDK. Para compilar un shader: `mgcb /build:MiShader.fx /outputDir:. /platform:Windows`. Genera MiShader.xnb que se coloca en Content/Effects/ del mod. La ventaja de MGCB es que tambien compila texturas, audio, y otros assets, no solo shaders.

34.3 Script de build automatizado

```
#!/bin/bash
# build_shaders.sh - Compila todos los .fx a .xnb

SHADER_DIR="Content/Effects"
OUTPUT_DIR="Content/Effects"

for fx_file in $SHADER_DIR/*.fx; do
    if [ -f "$fx_file" ]; then
        name=$(basename "$fx_file" .fx)
        echo "Compiling $name.fx..."
        mgcb /build:"$fx_file" /outputDir:"$OUTPUT_DIR" /platform:Windows
        if [ $? -ne 0 ]; then
            echo "ERROR: Failed to compile $name.fx"
            exit 1
        fi
    fi
done

echo "All shaders compiled successfully."
```

35. Lista Completa de Texturas Necesarias

35.1 Texturas built-in de la libreria

Asset	Tamaño	Formato	Uso
Default.png	32x32	PNG RGBA	Textura por defecto cuando no se especifica
WhitePixel.png	1x1	PNG RGBA	Pixel blanco para operaciones programaticas
SoftGlow.png	64x64	PNG RGBA	Particula suave radial para blending aditivo
Spark.png	16x16	PNG RGBA	Chispa estrella 4 puntas
Smoke.png	32x32	PNG RGBA	Puff de humo con transparencia
Trail.png	64x8	PNG RGBA	Degradado lineal para trails
Star.png	32x32	PNG RGBA	Estrella para efectos magicos
Ring.png	64x64	PNG RGBA	Anillo para shockwaves
Orb.png	32x32	PNG RGBA	Esfera con iluminacion 3D simulada
Vortex.png	64x64	PNG RGBA	Espiral para efectos de portal
Aura.png	128x128	PNG RGBA	Aura radial para accesorios
Slash.png	96x96	PNG RGBA	Crescent para slash effects

35.2 Texturas para shaders

Asset	Tamaño	Uso
Noise.png	128x128	Textura de ruido Perlin para distortion shader
Curve.png	256x1	Curva de interpolacion para fade custom
LUT.png	1024x32	Look-Up Table para color grading (32x32x32 LUT)

35.3 Texturas para UI interna

Asset	Tamaño	Uso
PanelBackground.png	128x128	Fondo de panel UI
PanelFrame.png	128x128	Marco de panel UI
Box.png	32x32	Caja basica (9-slice)
BoxHighlight.png	32x32	Caja en hover
Scrollbar.png	8x32	Barra de scroll vertical
Scrollbutton.png	16x16	Boton de scroll
Plus.png	16x16	Boton mas
Minus.png	16x16	Boton menos
X.png	16x16	Boton cerrar

35.4 Especificaciones tecnicas de cada textura

Todas las texturas deben cumplir estas especificaciones para ser cargadas correctamente por tModLoader: formato PNG RGBA 32-bit (con canal alpha), tamaño maximo 4096x4096 (recomendado $\leq 256 \times 256$ para optimizar memoria), fondo transparente (no color solido), naming case-sensitive (el nombre del archivo debe coincidir exactamente con el path en C#).

36. Lista Completa de Shaders Necesarios

36.1 Shaders core

Shader	Archivo .fx	Uso
ParticleSystem	ParticleSystem.fx	Render de quad particles con todos los atributos
InstancedShader	InstancedShader.fx	GPU instancing para miles de partículas
ShapeShader	ShapeShader.fx	Formas procedurales via SDF (UI interna)

36.2 Shaders de screen effect

Shader	Archivo .fx	Uso
BloomShader	BloomShader.fx	Bloom post-processing
ChromaticAberration	ChromaticAberration.fx	Aberracion cromatica
VignetteShader	VignetteShader.fx	Oscurecimiento de bordes
ShockwaveShader	ShockwaveShader.fx	Onda expansiva
DistortionShader	DistortionShader.fx	Distorsion por calor
ColorGradingShader	ColorGradingShader.fx	Color grading via LUT
RadialBlurShader	RadialBlurShader.fx	Desenfoco radial

37. Lista Completa de Sonidos Necesarios

37.1 Sonidos built-in

Sonido	Formato	Duracion	Uso
Pop.wav	WAV mono 44.1kHz	< 0.5s	Spawn de partícula pequeña
Whoosh.wav	WAV mono 44.1kHz	0.5-1s	Spawn de partícula grande (slash, explosion)
Impact.wav	WAV mono 44.1kHz	< 0.3s	Impacto de partícula con tile/NPC
Sparkle.wav	WAV mono 44.1kHz	0.3-0.5s	Spawn de partícula magica

37.2 Audio para efectos pre-construidos

Sonido	Formato	Uso
Explosion.wav	WAV stereo	ExplosionPreset
AuraLoop.wav	OGG stereo loop	AuraPreset (loop continuo)
Vortex.wav	WAV stereo	VortexPreset
TrailWhoosh.wav	WAV mono	TrailPreset

38. Generacion Procedural de Texturas con Python

38.1 Script para generar todas las texturas

En lugar de dibujar cada textura manualmente en Aseprite o Photoshop, muchas texturas base pueden generarse proceduralmente con Python + Pillow. Esto es especialmente util para texturas radialmente simetricas (SoftGlow, Star, Ring, Aura), degradados (Trail), y patrones matematicos (Vortex). Las texturas con forma organica (Smoke, Spark) requieren dibujo manual o uso de ruido Perlin.

38.2 Ejemplo: generar SoftGlow.png

```
from PIL import Image
import math

def make_soft_glow(size=64):
    img = Image.new("RGBA", (size, size), (0, 0, 0, 0))
    cx, cy = size / 2, size / 2
    max_r = size / 2

    for y in range(size):
        for x in range(size):
            dist = math.sqrt((x - cx) ** 2 + (y - cy) ** 2)
            if dist < max_r:
                t = dist / max_r
                alpha = int(255 * (1 - t) ** 3) # Falloff cubico
                img.putpixel((x, y), (255, 255, 255, alpha))

    img.save("SoftGlow.png")
    return img

make_soft_glow()
```

38.3 Ejemplo: generar Trail.png

```
def make_trail(width=64, height=8):
    img = Image.new("RGBA", (width, height), (0, 0, 0, 0))
    for x in range(width):
        t = x / width
        alpha = int(255 * (1 - t) ** 1.5)
        for y in range(height):
            img.putpixel((x, y), (255, 255, 255, alpha))
    img.save("Trail.png")
    return img

make_trail()
```

39. Diseno de Texturas por Tipo

39.1 Texturas radiales (SoftGlow, Aura, Ring)

Las texturas radiales tienen simetria circular: el centro es el punto mas brillante y el falloff hacia los bordes sigue una curva (lineal, cuadratica, cubica). El parametro clave es el "falloff exponent": exponente 1 da falloff lineal (suave), exponente 3 da falloff rapido (mas enfocado). Para partículas que se usan con blending aditivo, el fondo debe ser negro (RGB=0,0,0) porque additive suma colores y el negro suma 0.

39.2 Texturas con shape definida (Star, Spark, Slash)

Estas texturas tienen una forma geometrica definida (estrella, chispa, crescent) sobre fondo transparente. El reto es que la forma se vea bien a tamaños pequeños (16x16) sin perder definicion. Tecnicas: usar anti-aliasing en los bordes, evitar detalles finos que se pierdan al escalar, y mantener la silueta clara incluso a 1x scale.

39.3 Texturas con animacion (spritesheets)

Los spritesheets animados contienen multiples frames en una sola textura, tipicamente en fila horizontal. La cantidad de frames se especifica en `Main.projFrames[Type]` o via campo custom. Cada frame debe tener el mismo tamaño. La animacion se controla en `FindFrame()` del `ModProjectile`.

40. Sistema de Configuracion del Mod

40.1 ModConfig de tModLoader

tModLoader permite crear configuraciones que el usuario puede ajustar in-game via ModConfig. Esto es util para librerias que quieren ofrecer opciones de performance o calidad visual. La configuracion se serializa automaticamente y se guarda en un archivo JSON.

```
using Terraria.ModLoader.Config;
using System.ComponentModel;

namespace MiParticleLib
{
    public class ParticleConfig : ModConfig
    {
        public override ConfigScope Mode => ConfigScope.ClientSide;

        [Header("Performance")]
        [DefaultValue(5000)]
        [Range(500, 50000)]
        [Slider]
        public int MaxParticles { get; set; } = 5000;

        [DefaultValue(true)]
        public bool EnableCulling { get; set; } = true;

        [DefaultValue(false)]
        public bool EnableGPUParticles { get; set; } = false;

        [Header("Visuals")]
        [DefaultValue(1.0f)]
        [Range(0.1f, 2.0f)]
        [Slider]
        public float GlobalParticleIntensity { get; set; } = 1.0f;

        [DefaultValue(true)]
        public bool EnableAdditiveBlending { get; set; } = true;

        [DefaultValue(false)]
        public bool EnableScreenShake { get; set; } = false;

        [Header("Debug")]
        [DefaultValue(false)]
        public bool ShowDebugOverlay { get; set; } = false;

        [DefaultValue(false)]
        public bool VerboseLogging { get; set; } = false;

        public override void OnChanged()
        {
            // Se llama cuando el usuario cambia un valor
            if (MiParticleLib.Instance != null)
            {
            }
        }
    }
}
```

```

        {
            // Re-inicializar si cambio MaxParticles
            ParticleManager.ReinitializeIfNeeded(MaxParticles);
        }
    }
}

```

41. Sistema de Logging

41.1 Por que logging estructurado

Una libreria de particulas maneja miles de instancias y operaciones por frame. Cuando algo falla, los `Console.WriteLine` o `Main.NewText` no escalan. `tModLoader` expone `Mod.Logger` que escribe al archivo de log con timestamp y nivel. Para mensajes que el usuario debe ver en chat in-game, se usan los metodos estaticos `LogPublicInfo`, `LogPublicWarning` y `LogPublicError` (de la clase `Logging` de `tModLoader 1.4.4`). La combinacion de ambos es lo correcto: `Mod.Logger` para detalle tecnico en archivo, `LogPublic*` para alertas visibles al usuario.

```

public static class ParticleLogger
{
    private static Mod _mod;
    private static bool _verbose;

    public static void Initialize(Mod mod, bool verbose)
    {
        _mod = mod;
        _verbose = verbose;
    }

    public static void Info(string message)
    {
        _mod?.Logger.Info($"[ParticleLib] {message}");
    }

    public static void Warn(string message)
    {
        _mod?.Logger.Warn($"[ParticleLib] {message}");
    }

    public static void Error(string message, Exception ex = null)
    {
        _mod?.Logger.Error($"[ParticleLib] {message}" + (ex != null ? $"{\n{ex}" :
""));
    }

    public static void Debug(string message)
    {
        if (_verbose)
            _mod?.Logger.Info($"[ParticleLib DEBUG] {message}");
    }

    public static void Stats()
    {
        if (!_verbose) return;
        Info($"Active: {ParticleManager.ActiveCount}/{ParticleManager.Capacity}");
    }
}

```

```
}
```

42. Debugging y Profiling

42.1 Debug overlay

Para desarrollo, es útil tener un overlay que muestre estadísticas en pantalla: partículas activas, capacity, FPS, memoria usada. Esto se implementa via `PostDrawInterface` y dibuja texto con `Main.spriteBatch.DrawString`.

```
public class DebugOverlay : ModSystem
{
    public override void PostDrawInterface()
    {
        if (!MiParticleLib.Config.ShowDebugOverlay) return;

        var sb = Main.spriteBatch;
        sb.Begin();

        int y = 100;
        DrawLine(sb, $"Particles:
{ParticleManager.ActiveCount}/{ParticleManager.Capacity}", ref y);
        DrawLine(sb, $"FPS: {Main.frameRate}", ref y);
        DrawLine(sb, $"GC Memory: {GC.GetTotalMemory(false) / 1024} KB", ref y);
        DrawLine(sb, $"Active Textures: {TextureRegistry.Count}", ref y);

        sb.End();
    }

    private void DrawLine(SpriteBatch sb, string text, ref int y)
    {
        sb.DrawString(Main.fontMouseText, text, new Vector2(10, y), Color.White);
        y += 20;
    }
}
```

42.2 Profiling con Stopwatch

Para medir cuanto tarda cada parte del pipeline, se usa `System.Diagnostics.Stopwatch`. Se mide `Update` y `Render` por separado, y se promedia sobre `N` frames para obtener un número estable.

```
using System.Diagnostics;

public class ParticleProfiler
{
    private static Stopwatch _updateSW = new();
    private static Stopwatch _renderSW = new();
    private static long _updateTotalMs;
    private static long _renderTotalMs;
    private static int _frameCount;
    private const int SampleFrames = 60;

    public static double AvgUpdateMs { get; private set; }
    public static double AvgRenderMs { get; private set; }

    public static void BeginUpdate() => _updateSW.Restart();
    public static void EndUpdate()
```

```

    {
        _updateSW.Stop();
        _updateTotalMs += _updateSW.ElapsedMilliseconds;
    }

    public static void BeginRender() => _renderSW.Restart();
    public static void EndRender()
    {
        _renderSW.Stop();
        _renderTotalMs += _renderSW.ElapsedMilliseconds;
    }

    public static void EndFrame()
    {
        _frameCount++;
        if (_frameCount >= SampleFrames)
        {
            AvgUpdateMs = (double)_updateTotalMs / _frameCount;
            AvgRenderMs = (double)_renderTotalMs / _frameCount;
            _updateTotalMs = 0;
            _renderTotalMs = 0;
            _frameCount = 0;
        }
    }
}

```

43. Testing en Multiplayer

43.1 Setup de test multiplayer

Para testear multiplayer sin una segunda computadora, se pueden iniciar dos instancias de tModLoader: una como host (via Steam) y otra como guest (tambien via Steam). Ambas instancias deben tener el mod instalado y los mismos config. Los bugs mas comunes en multiplayer son: desync de estado (el servidor y el cliente ven cosas diferentes), crashes por `Main.player[i]` null, y hooks que se ejecutan en servidor sin chequear `Main.netMode`.

43.2 Checklist de multiplayer-safe

- Toda llamada a `ParticleManager.Spawn` debe chequear `Main.netMode != 2`
- Toda llamada a `ParticleManager.Render` debe chequear `Main.dedicatedServer == false`
- No usar `Main.LocalPlayer` en hooks que se ejecutan en servidor (usar `player parameter`)
- No usar variables static para estado de instancia (causa desync entre clientes)
- Para datos custom de jugador, usar `ModPlayer` con fields syncable
- Para eventos que deben verse en todos los clientes, usar `ModPacket`

44. Performance Benchmarks

44.1 Metricas objetivo

Para una libreria de particulas, las metricas objetivo razonables en hardware moderno son: 10000 partículas activas a 60 FPS en una CPU i5, spawn de 1000 partículas por frame sin hitches, render de 5000 partículas visibles en menos de 2ms por frame, y uso de memoria menor a 50 MB para el buffer principal.

44.2 Tabla de benchmarks esperados

Configuracion	Particulas activas	FPS esperado	Memoria
Minimo (low-end)	500	60	~10 MB
Recomendado (mid)	5000	60	~30 MB
Alto (high-end)	10000	60	~50 MB
Maximo (enthusiast)	50000	30-60	~150 MB

45. Cross-Mod Compatibility

45.1 Mod Calls API

Para que otros mods puedan usar tu libreria sin depender fuertemente de ella, se expone una API via Mod.Call. El mod destino define un metodo Call(object[] args) que recibe argumentos y devuelve un object. Esto permite cross-mod sin modReferences.

```
public override object Call(params object[] args)
{
    if (args.Length == 0) return null;

    string command = args[0] as string;
    switch (command)
    {
        case "SpawnParticle":
            return SpawnParticleFromCall(args);

        case "SpawnShape":
            return SpawnShapeFromCall(args);

        case "RegisterTexture":
            return RegisterTextureFromCall(args);

        case "GetStats":
            return new { Active = ParticleManager.ActiveCount, Capacity = ParticleManager.Capacity };

        default:
            ParticleLogger.Warn($"Unknown Call command: {command}");
            return null;
    }
}
```

```

    }
}

private bool SpawnParticleFromCall(object[] args)
{
    // args[1] = position (Vector2)
    // args[2] = velocity (Vector2)
    // args[3] = color (Color)
    // args[4] = scale (float)
    // args[5] = duration (int)
    // args[6] = texturePath (string)
    if (args.Length < 7) return false;

    Vector2 pos = (Vector2)args[1];
    Vector2 vel = (Vector2)args[2];
    Color col = (Color)args[3];
    float scale = (float)args[4];
    int duration = (int)args[5];
    string texPath = (string)args[6];

    ushort texId = TextureRegistry.Register(texPath);

    var data = new ParticleData
    {
        Position = pos,
        Velocity = vel,
        Scale = new Vector2(scale),
        PackedColor = PackColor(col),
        TimeLeft = duration,
        Duration = duration,
        TextureId = texId,
    };

    return ParticleManager.Spawn(data);
}

```

45.2 WeakReferences

Para mods que quieren usar tu libreria pero tambien funcionar sin ella, se usa weakReferences en build.txt. Esto permite que el mod cargue sin tu libreria, pero activa el contenido cross-mod cuando tu libreria esta presente.

46. Documentacion de la API Publica

46.1 XML documentation comments

Cada clase y metodo publico debe tener XML doc comments. Estos se compilan a un archivo .xml que Visual Studio usa para IntelliSense. Es la forma estandar de documentar APIs en C#.

```

/// <summary>
/// Spawnea una partícula individual con los datos especificados.
/// </summary>
/// <param name="data">Los datos de la partícula a spawnear.</param>
/// <returns>
/// true si la partícula se spawneo exitosamente;
/// false si el buffer esta lleno.
/// </returns>

```

```

/// <example>
/// <code>
/// var data = new ParticleData {
///     Position = player.Center,
///     Velocity = Vector2.Zero,
///     Scale = Vector2.One,
///     Color = Color.Red,
///     TimeLeft = 60,
///     Duration = 60,
///     TextureId = TextureRegistry.SoftGlow
/// };
/// ParticleManager.Spawn(data);
/// </code>
/// </example>
public static bool Spawn(ParticleData data)

```

46.2 API publica principal

Clase	Metodo	Descripcion
ParticleManager	Spawn(ParticleData)	Spawnea una partícula individual
ParticleManager	SpawnShape(center, shape, count, template)	Spawnea N partículas en una shape
ParticleManager	Clear()	Desactiva todas las partículas
TextureRegistry	Register(path)	Registra una textura y devuelve su ID
TextureRegistry	Get(id)	Obtiene la Texture2D por ID
Emitter (base)	Update()	Actualiza el emitter (override en subclasses)
ShapeDescriptor	Box(w, h)	Crea shape de caja solida
ShapeDescriptor	Circle(r)	Crea shape de circulo solido
ShapeDescriptor	HollowBox(w, h)	Crea shape de caja hueca
ShapeDescriptor	HollowCircle(r)	Crea shape de circulo hueco
ShapeDescriptor	Cone(length, angle)	Crea shape de cono
ShapeDescriptor	Sphere(r)	Crea shape de esfera 3D proyectada
ShapeDescriptor	Vortex(r, turns)	Crea shape de vortex
ComponentFlag	Gravity, FadeOut, etc.	Bitmasks para activar components
ParticleData	EnableComponent(flag)	Activa un componente en la partícula
LayerPriority	BeforeNPCs, etc.	Constantes de layer numericas

47. Publicacion en Steam Workshop

47.1 Preparacion para publicar

Antes de publicar, verificar: build.txt tiene version correcta, description.txt es claro y atribuido, icon.png es 256x256, no hay errores de build, y el mod ha sido testeado en multiplayer. Tambien es buena practica tener un GitHub repo publico con el source.

47.2 Proceso de publicacion

En tModLoader in-game: Workshop -> Develop Mods -> seleccionar mod -> click "Publish" -> configurar tags (Library, Utilities, VFX) -> escribir descripcion con formato Markdown -> subir screenshots -> click "Publish". El mod recibira un Workshop ID unico.

47.3 Update workflow

Para actualizar: bump de version en build.txt (OBLIGATORIO), rebuild del mod, click "Update" en el menu Workshop. Steam Workshop rechaza updates sin bump de version. La convencion es: PATCH para bugfixes, MINOR para nuevo contenido, MAJOR para breaking changes.

48. Mantenimiento y Versionado

48.1 Semantic versioning

Seguir semver.org: MAJOR.MINOR.PATCH. MAJOR para cambios incompatibles (rompen mods que dependen de tu libreria). MINOR para nuevo contenido compatible hacia atras. PATCH para bugfixes. Cada release debe tener un CHANGELOG.md actualizado.

48.2 Backwards compatibility

Para una libreria usada por otros mods, mantener backwards compatibility es critico. Si se cambia una API publica, marcar la antigua como [Obsolete] con mensaje indicando la nueva. Dar al menos una version MINOR de gracia antes de eliminar la API obsoleta.

48.3 Branching strategy

Git branching recomendada: main (estable, solo releases), dev (desarrollo activo), feature/xyz (features especificos), hotfix/xyz (fixes urgentes). Hacer PR de feature a dev, y de dev a main cuando se publica release.

49. Roadmap y Features Futuras

49.1 Features planeadas para v0.2

- Texture atlas automatico para reducir draw calls
- Mas components built-in: Orbit, Bounce, DieOnTile, EmitLight
- Sistema de presets: Explosion, Aura, Vortex, Trail pre-empaquetados
- Editor visual in-game para crear y testear efectos

49.2 Features planeadas para v0.3

- GPU particles via instancing (10000+ en un draw call)
- Trail system integrado (no via ModProjectile)
- Sistema de colision con tiles optimizado
- Spatial partitioning (grid-based) para queries por area

49.3 Features planeadas para v1.0

- Cross-platform testing (Windows, Mac, Linux)
- Localizacion es/pt/fr/de
- Tutorial interactivo integrado
- Performance dashboard web (lectura de logs)
- Integration con Mod Browser para descargar presets de comunidad

50. Apendice A: Snippets Completos deCodigo

50.1 Spawn de explosion basica

```
public static void SpawnExplosion(Vector2 center, float radius = 30f, int count = 20)
{
    if (Main.dedicatedServer) return;

    var template = new ParticleData
    {
        Velocity = Vector2.Zero,
        Scale = Vector2.One * 1.5f,
        PackedColor = PackColor(Color.Orange),
        PackedStartColor = PackColor(Color.Yellow),
        PackedEndColor = PackColor(Color.Red),
        TimeLeft = 45,
        Duration = 45,
        TextureId = TextureRegistry.SoftGlow,
        BlendMode = 1, // Additive
        LayerPriority = LayerPriorities.BeforeProjectiles,
    };
    template.EnableComponent(ComponentFlag.FadeOut);
    template.EnableComponent(ComponentFlag.ScaleDown);
    template.EnableComponent(ComponentFlag.ColorShift);

    ParticleManager.SpawnShape(center, ShapeDescriptor.Circle(radius), count, template);

    // Screen shake
    Main.instance.CameraModifiers.Add(new PunchCameraModifier(
        center, new Vector2(1, 0), 8f, 10, 20, 0.4f, "Explosion"));
}
```

50.2 Aura orbital persistente

```
public class AuraPreset
{
    public static void SpawnOrbitalParticle(Vector2 center, Color color, float
radius = 40f)
    {
        if (Main.dedicatedServer) return;

        float angle = Main.rand.NextFloat(0f, MathHelper.TwoPi);
        Vector2 offset = new Vector2(radius, 0f).RotatedBy(angle);

        var data = new ParticleData
        {
            Position = center + offset,
            Velocity = Vector2.Zero,
            Scale = Vector2.One * 0.7f,
            PackedColor = PackColor(color),
            TimeLeft = 60,
            Duration = 60,
            TextureId = TextureRegistry.SoftGlow,
            BlendMode = 1,
            LayerPriority = LayerPriorities.BeforePlayers,
        };
        data.EnableComponent(ComponentFlag.FadeOut);
        data.UserData0 = radius; // Para OrbitComponent
        data.UserData1 = angle;
        data.EnableComponent(ComponentFlag.Orbit);

        ParticleManager.Spawn(data);
    }
}
```

51. Apendice B: Lista de IDs de Terraria Utiles

51.1 ItemUseStyleID

ID	Nombre	Uso
1	Swing	Espadas, hachas, pickaxes
2	Shoot	Arcos, guns, varitas magicas
3	Thrust	Lanzas cortas
4	HoldUp	Items rituales, summon items
5	EatFood	Comida
6	DrinkLiquid	Pociones (nombre exacto en Terraria decompilado)
7	Hold	Antorcha, items que se mantienen
8	Fist	Puños, claws
9	Yoyo	Yoyos (con channel=true)
10	Whip	Whips
11	Mow	Segadora
12	Throw	Items arrojados

51.2 SoundID utiles

ID	Sonido
1	Espada swing
5	Arco shoot
11	Gun shot
20	Hit NPC
44	Summon minion
152	Whip
169	Magic cast

52. Apendice C: Glosario

Termino	Definicion
AoS	Array of Structures - cada elemento del array es un struct completo
API	Application Programming Interface - contrato publico de una libreria
Behavior	Patron donde una clase define el comportamiento de entidades
Blending	Como se combinan colores al dibujar (alpha, additive, etc.)
Buffer	Region de memoria pre-asignada para almacenar datos
Cache locality	Acceso a datos contiguos en memoria, mejora rendimiento
Component	Unidad de comportamiento reutilizable en ECS
Culling	No dibujar objetos fuera de pantalla (frustum culling)
DOD	Data-Oriented Design - paradigma orientado a datos
ECS	Entity-Component-System - patron arquitectonico
Emitter	Objeto que genera partículas en una shape
GC	Garbage Collector - recolector de memoria de C#
GPL v3	Licencia copyleft fuerte
HLSL	High-Level Shading Language - lenguaje de shaders de DirectX
Hook	Punto de extension donde se puede inyectar codigo custom
Instancing	Renderizar multiples instancias en un draw call
Layer	Capa de render, controla el orden de dibujado
LUT	Look-Up Table - textura que mapea colores de entrada a salida
MIT	Licencia permisiva
ModConfig	Sistema de configuracion de tModLoader
ModSystem	Clase base para sistemas globales del mod
OOP	Object-Oriented Programming
Premultiplied alpha	Formato donde el RGB se multiplica por el alpha
Round-robin	Estrategia que rota entre elementos para distribuir carga
SDF	Signed Distance Field - tecnica para formas procedurales
Shader	Programa que se ejecuta en el GPU
SoA	Structure of Arrays - cada campo en array separado

Termino	Definicion
SpriteBatch	API de MonoGame para dibujar texturas 2D
tModLoader	API de modding para Terraria
Vertex format	Estructura de datos que define los atributos de un vertice
VFX	Visual Effects
XNB	Formato binario de assets de MonoGame/XNA

53. Apendice D: Recursos y Referencias

53.1 Documentacion oficial

Recurso	URL
tModLoader API docs	docs.tmodloader.net
tModLoader GitHub	github.com/tModLoader/tModLoader
tModLoader Wiki	github.com/tModLoader/tModLoader/wiki
Terraria Wiki	terraria.wiki.gg
MonoGame docs	docs.monogame.net
HLSL reference	learn.microsoft.com/en-us/windows/win32/direct3dhlsl/dx-graphics-hlsl

53.2 Librerias de referencia

Libreria	URL
ParticleLibrary (SnowyStarfall)	github.com/SnowyStarfall/ParticleLibrary
ParticleAnimationLib (Emafire003)	github.com/Emafire003/ParticleAnimationLib
TerrariaOverhaul (Mirsario)	github.com/Mirsario/TerrariaOverhaul
AnimLib (Ilemni)	github.com/Ilemni/AnimLib
Luminance (LucilleKarma)	github.com/LucilleKarma/Luminance
CalamityModPublic	github.com/CalamityTeam/CalamityModPublic

53.3 Comunidad

Recurso	URL
Foro Terraria mods	forums.terraria.org
Reddit r/Terraria	reddit.com/r/Terraria
Reddit r/terrariamods	reddit.com/r/terrariamods
Discord tModLoader	discord.gg/tmodloader
Discord ParticleLibrary	discord.gg/bhCuppRdVF

53.4 Herramientas

Herramienta	URL	Para que
Visual Studio Community	visualstudio.microsoft.com	IDE para C#
.NET SDK	dotnet.microsoft.com	Compilador C#
MonoGame SDK	monogame.net	Framework + MGCB
Aseprite	aseprite.org	Pixel art
Audacity	audacityteam.org	Edicion de audio
Git	git-scm.com	Version control

53.5 Conclusion final

Este documento ha cubierto exhaustivamente todos los aspectos de crear una libreria de particulas para Terraria desde cero: analisis de soluciones existentes, decisiones de arquitectura, implementacion del nucleo (structs, buffers, components, emitters, manager), integracion con tModLoader via hooks, asset pipeline (texturas y shaders), 10 shaders HLSL completos con codigo fuente, lista exhaustiva de assets necesarios, generacion procedural de texturas, sistema de configuracion, logging, debugging, profiling, testing multiplayer, performance benchmarks, cross-mod compatibility, documentacion de API, publicacion en Steam Workshop, mantenimiento y roadmap. La IA que use este documento como contexto deberia tener todo el conocimiento necesario para generar codigo funcional de la libreria sin asistencia adicional. Para preguntas especificas, referirse a los capitulos correspondientes o a los recursos listados en este apendice.